

Comprehensive Guide to Software Design Principles

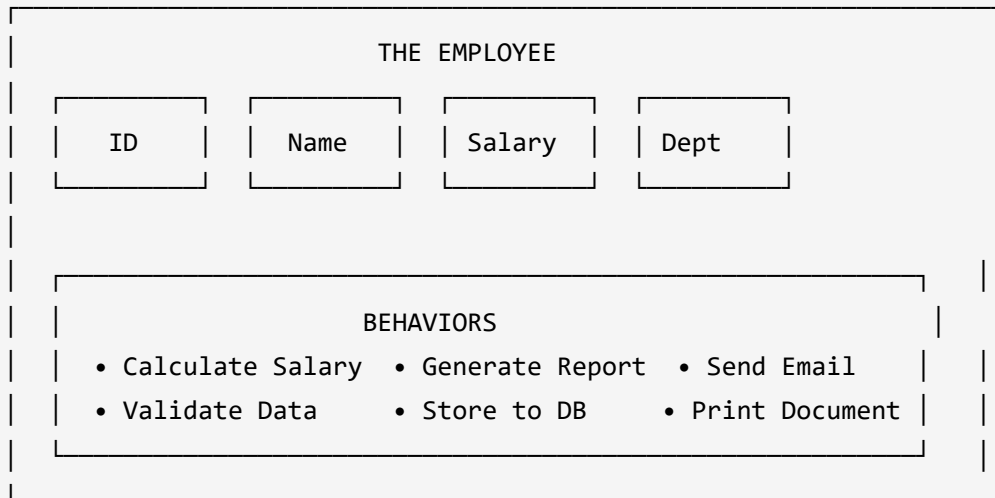
Using C++ and the Employee Class

Table of Contents

1. [SOLID Principles](#)
 - [S: Single Responsibility Principle \(SRP\)](#)
 - [O: Open/Closed Principle \(OCP\)](#)
 - [L: Liskov Substitution Principle \(LSP\)](#)
 - [I: Interface Segregation Principle \(ISP\)](#)
 - [D: Dependency Inversion Principle \(DIP\)](#)
 2. [DRY Principle](#)
 3. [KISS Principle](#)
 4. [Practical Application](#)
 5. [Designing for Scalability and Maintainability](#)
 - [a. Scalability in Design](#)
 - [b. Maintainability in Design](#)
 - [c. Patterns and Techniques](#)
-

Introduction

Throughout this document, we'll use the **Employee** class as our primary example to illustrate each principle. Think of the Employee class as a real employee in a company — they have a name, ID, salary, and various responsibilities.



PART 1: SOLID PRINCIPLES

The SOLID Acronym

S	O	L	I	D
▼	▼	▼	▼	▼
Single	Open	Liskov	Interface	Dependency
Responsibility	Closed		Segregation	Inversion
Principle	Principle	Principle	Principle	Principle

SOLID helps you write code that is:

- **Maintainable** — easy to change
- **Extensible** — easy to add features
- **Testable** — easy to unit test
- **Readable** — easy to understand

1. S: Single Responsibility Principle (SRP)

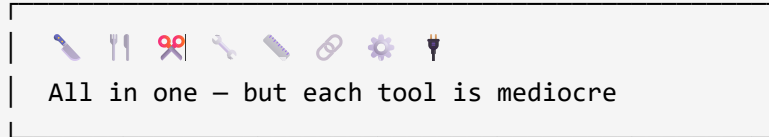
Definition

"A class should have only one reason to change."

The Analogy

Think of a **Swiss Army Knife** vs a **specialized tool**.

❌ BAD: Swiss Army Knife (Does everything poorly)



✅ GOOD: Specialized Tools (Each does one thing great)

			
Knife	Scissors	Screwdrv	Ruler
Cut well	Cut well	Tighten	Measure

A Swiss Army Knife tries to do everything, but a professional chef uses specialized knives. Similarly, a class should do one thing well.

Bad Example (Violates SRP)

```
#include <iostream>
#include <string>
#include <fstream>
#include <vector>

class Employee {
private:
    std::string name;
    int id;
    double salary;

public:
    Employee(std::string n, int i, double s)
        : name(n), id(i), salary(s) {}

    // ✖ VIOLATION: Multiple reasons to change!

    // Responsibility 1: Data storage
    void saveToFile() {
        std::ofstream file("employee_" + std::to_string(id) + ".txt");
        file << "Name: " << name << "\n";
        file << "ID: " << id << "\n";
        file << "Salary: " << salary << "\n";
        file.close();
    }

    // Responsibility 2: Printing/Formatting
    void printReport() {
        std::cout << "=== Employee Report ===\n";
    }
}
```

```
std::cout << "Name: " << name << "\n";
std::cout << "ID: " << id << "\n";
std::cout << "Salary: $" << salary << "\n";
std::cout << "=====\n";
}

// Responsibility 3: Database operations
void saveToDatabase() {
    // Database logic here...
    std::cout << "Saving to database...\n";
}

// Responsibility 4: Email notifications
void sendEmail() {
    std::cout << "Sending email notification...\n";
}

// Responsibility 5: XML export
void exportToXML() {
    std::cout << "<employee><name>" << name << "</name></employee>\n";
}

// ... more responsibilities could be added forever!
};
```

Why this is bad:

- If the file format changes, we modify Employee
- If the report format changes, we modify Employee
- If the database schema changes, we modify Employee
- If the email system changes, we modify Employee

- Testing becomes difficult (all dependencies are mixed)

Good Example (Follows SRP)

```
#include <iostream>
#include <string>
#include <fstream>
#include <vector>

// ✅ GOOD: Each class has ONE responsibility

// ===== CORE EMPLOYEE CLASS =====
class Employee {
private:
    std::string name;
    int id;
    double salary;

public:
    Employee(std::string n, int i, double s)
        : name(n), id(i), salary(s) {}

    // Core responsibility: Store employee data
    std::string getName() const { return name; }
    int getId() const { return id; }
    double getSalary() const { return salary; }
};

// ===== FILE STORAGE =====
class EmployeeFileStorage {
public:
    void save(const Employee& emp) {
        std::string filename = "employee_" + std::to_string(emp.getId()) + ".txt";
```



```
        std::ofstream file(filename);
        file << "Name: " << emp.getName() << "\n";
        file << "ID: " << emp.getId() << "\n";
        file << "Salary: " << emp.getSalary() << "\n";
        file.close();
        std::cout << "Saved to file: " << filename << "\n";
    }
};

// ===== REPORT GENERATION =====
class EmployeeReportGenerator {
public:
    void generateReport(const Employee& emp) {
        std::cout << "=== Employee Report ===\n";
        std::cout << "Name: " << emp.getName() << "\n";
        std::cout << "ID: " << emp.getId() << "\n";
        std::cout << "Salary: $" << emp.getSalary() << "\n";
        std::cout << "=====\n";
    }
};

// ===== EMAIL NOTIFICATION =====
class EmployeeEmailService {
public:
    void sendNotification(const Employee& emp) {
        std::cout << "Email sent to: " << emp.getName() << "\n";
    }
};

// ===== XML EXPORT =====
class EmployeeXMLExporter {
public:
```

```
void exportToXML(const Employee& emp) {
    std::cout << "<employee>\n";
    std::cout << "  <name>" << emp.getName() << "</name>\n";
    std::cout << "  <id>" << emp.getId() << "</id>\n";
    std::cout << "  <salary>" << emp.getSalary() << "</salary>\n";
    std::cout << "</employee>\n";
}

};

// ===== DEMONSTRATION =====
int main() {
    Employee emp("John Doe", 101, 50000.0);

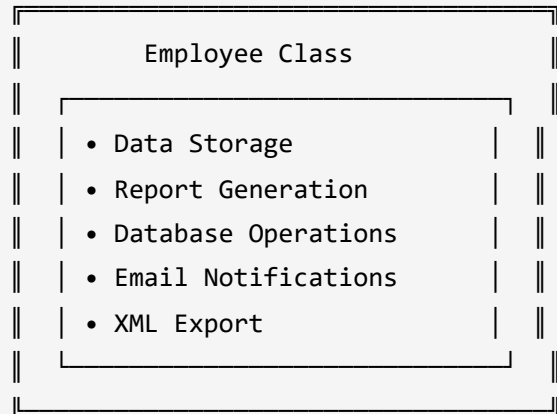
    EmployeeFileStorage fileStorage;
    EmployeeReportGenerator reportGen;
    EmployeeEmailService emailService;
    EmployeeXMLExporter xmlExporter;

    // Each component does ONE thing
    fileStorage.save(emp);
    reportGen.generateReport(emp);
    emailService.sendNotification(emp);
    xmlExporter.exportToXML(emp);

    return 0;
}
```

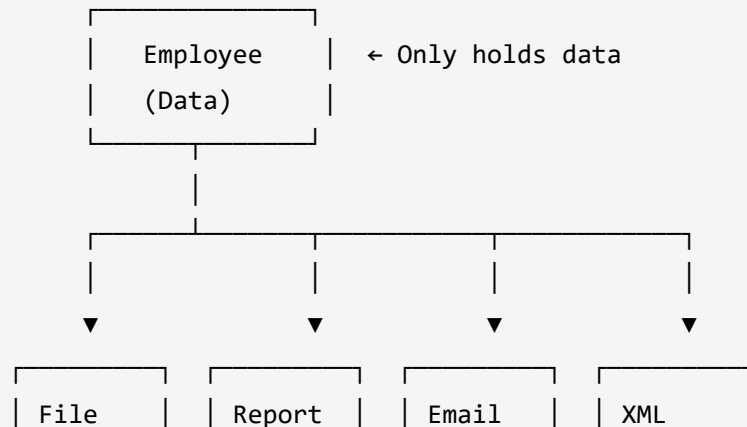
Visual Representation

BAD DESIGN (One class does everything)



|
| Change in any requirement
| forces changes to Employee
▼

GOOD DESIGN (Each class does ONE thing)





Each can change independently!

When to Apply SRP

Symptom	What It Means	Solution
Class is too large	Multiple responsibilities	Split into smaller classes
Changes to one feature break another	Responsibilities are coupled	Separate concerns
Class is hard to test	Too many dependencies	Split and use dependency injection
You can't explain class purpose in one sentence	Has multiple reasons to change	Refactor to single responsibility

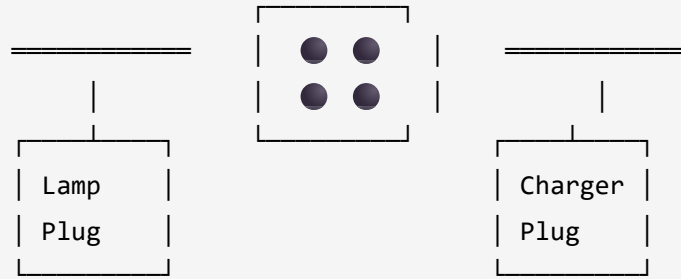
2. O: Open/Closed Principle (OCP)

Definition

"Software entities should be open for extension, closed for modification."

The Analogy

Think of a **Power Outlet** in your home.



The outlet is OPEN (accepts any plug)

The outlet is CLOSED (you don't modify it to add new devices)

The wall outlet doesn't need to be modified every time you buy a new device. You extend its capability by plugging in new devices.

Bad Example (Violates OCP)

```
#include <iostream>
#include <string>
#include <vector>

class Employee {
public:
    std::string name;
    double baseSalary;
    std::string role;

    Employee(std::string n, double salary, std::string r)
        : name(n), baseSalary(salary), role(r) {}
};

// ✗ VIOLATION: Every time we add a new role, we must modify this function
double calculateBonus(Employee& emp) {
    if (emp.role == "Manager") {
        return emp.baseSalary * 0.20; // 20% bonus
    } else if (emp.role == "Engineer") {
        return emp.baseSalary * 0.15; // 15% bonus
    } else if (emp.role == "Intern") {
        return emp.baseSalary * 0.05; // 5% bonus
    } else if (emp.role == "Director") { // <-- New role added!
        return emp.baseSalary * 0.25; // 25% bonus
    }
    // Every new role requires modifying this function!
    return 0;
}
```

Problem: Every time a company adds a new role, we must modify `calculateBonus()` . This risks breaking existing functionality.

Good Example (Follows OCP)

```
#include <iostream>
#include <string>
#include <vector>

// ✅ Base class - defines the interface
class Employee {
public:
    std::string name;
    double baseSalary;

    Employee(std::string n, double salary)
        : name(n), baseSalary(salary) {}

    // Virtual method - each role calculates bonus differently
    virtual double calculateBonus() const = 0;
    virtual ~Employee() = default;
};

// ✅ Each role EXTENDS the behavior without modifying base code

class Manager : public Employee {
public:
    Manager(std::string n, double salary) : Employee(n, salary) {}

    double calculateBonus() const override {
        return baseSalary * 0.20; // 20% bonus
    }
};
```



```
class Engineer : public Employee {
public:
    Engineer(std::string n, double salary) : Employee(n, salary) {}

    double calculateBonus() const override {
        return baseSalary * 0.15; // 15% bonus
    }
};

class Intern : public Employee {
public:
    Intern(std::string n, double salary) : Employee(n, salary) {}

    double calculateBonus() const override {
        return baseSalary * 0.05; // 5% bonus
    }
};

// ✅ Adding a new role? Just add a new class - NO modifications to existing code!
class Director : public Employee {
public:
    Director(std::string n, double salary) : Employee(n, salary) {}

    double calculateBonus() const override {
        return baseSalary * 0.30; // 30% bonus
    }
};

// ✅ This function works for ALL employee types - NO modification needed!
void printBonusReport(const Employee& emp) {
    std::cout << emp.name << " (Bonus: $" << emp.calculateBonus() << ")\n";
}
```

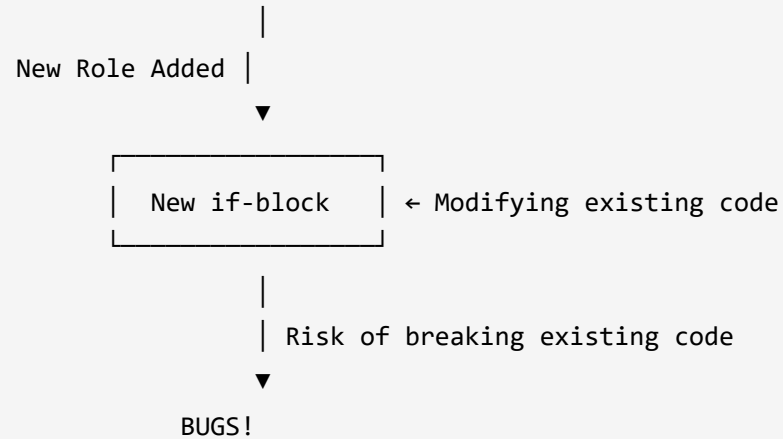
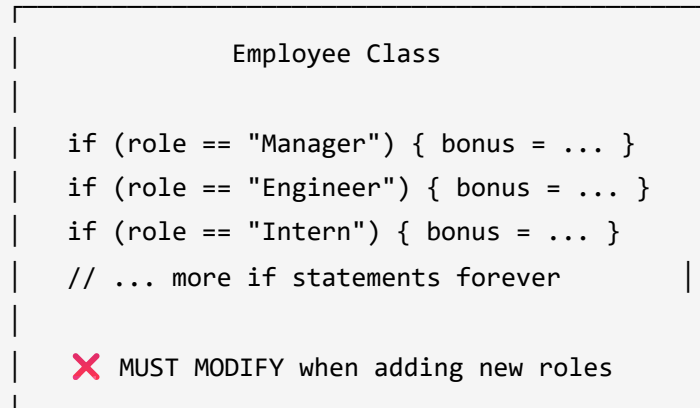
```
// ===== DEMONSTRATION =====  
  
int main() {  
    std::vector<Employee*> employees;  
  
    employees.push_back(new Manager("Alice", 80000));  
    employees.push_back(new Engineer("Bob", 70000));  
    employees.push_back(new Intern("Charlie", 30000));  
    employees.push_back(new Director("Diana", 100000)); // New role added easily!  
  
    std::cout << "=== Bonus Report ===\n";  
    for (const auto& emp : employees) {  
        printBonusReport(*emp);  
    }  
  
    // Clean up  
    for (auto emp : employees) {  
        delete emp;  
    }  
  
    return 0;  
}
```

Output:

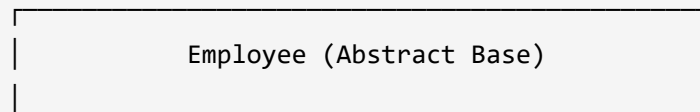
```
=== Bonus Report ===  
Alice (Bonus: $16000)  
Bob (Bonus: $10500)  
Charlie (Bonus: $1500)  
Diana (Bonus: $30000)
```

Visual Representation

BAD DESIGN (Open for Modification)

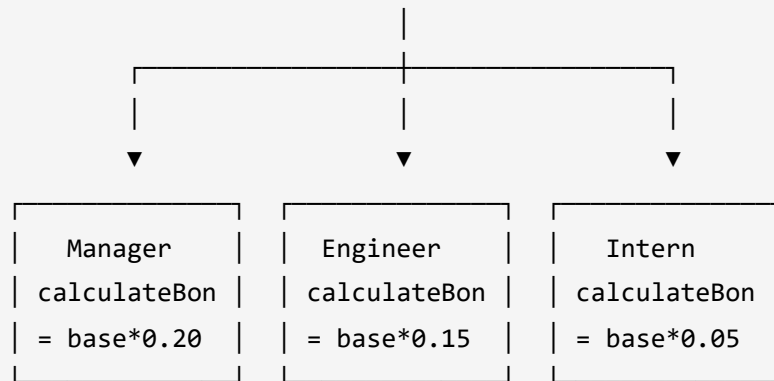


GOOD DESIGN (Open for Extension, Closed for Modification)



```
virtual double calculateBonus() = 0;
```

✓ CLOSED for modification
✓ OPEN for extension via inheritance



New Role? Just add a new class!

Director ← New class, no modification

Key Takeaway

OCP PRINCIPLE

DON'T: Add new behavior by modifying existing code

DO: Add new behavior by creating new classes

Old code: "If it's a Manager, do X"

New code: "Ask the object to do X" (Polymorphism)

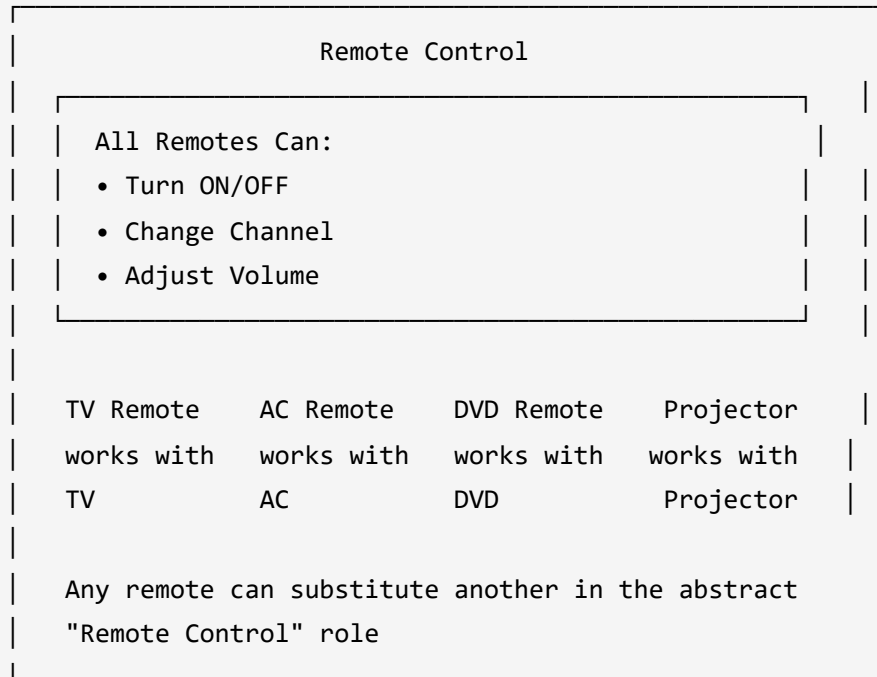
3. L: Liskov Substitution Principle (LSP)

Definition

"Objects of a superclass should be replaceable with objects of its subclasses without breaking the application."

The Analogy

Think of **Remote Controls**.



If a remote says it can "turn on the device," it must actually work. A "Smart Remote" that claims to be a remote but can't turn on devices violates LSP.

Bad Example (Violates LSP)

```
#include <iostream>
#include <string>

// ❌ VIOLATION: Subclass breaks the contract

class Employee {
public:
    virtual void work() {
        std::cout << "Employee is working...\n";
    }

    virtual void takeBreak() {
        std::cout << "Employee is taking a break.\n";
    }

    virtual double getSalary() {
        return 5000.0;
    }
};

class Intern : public Employee {
public:
    // ⚠️ PROBLEM: Intern doesn't get paid!
    // This violates the expectation that getSalary() returns actual salary
    double getSalary() override {
        return 0.0; // Interns work for free...
    }
};
```

```
class Robot : public Employee {
public:
    // ⚠️ PROBLEM: Robots don't take breaks!
    // This violates the contract
    void takeBreak() override {
        // Robots don't take breaks... but this is unexpected
        std::cout << "Robot does not take breaks!\n";
    }
};

// This function assumes ALL employees can take breaks
void organizeTeamBuilding(Employee& emp) {
    std::cout << "Organizing team building for: ";
    emp.takeBreak(); // What if emp is a Robot?
}

int main() {
    Intern intern("Charlie", 0); // Intern with 0 salary

    // Problem: This employee never gets paid!
    std::cout << "Intern's salary: $" << intern.getSalary() << "\n";
    // Output: $0.00 - Is this really an "Employee" in the financial sense?

    // Another problem:
    Robot robot("R2-D2");
    organizeTeamBuilding(robot);
    // Output: "Robot does not take breaks!" - This breaks the expectation

    return 0;
}
```


Good Example (Follows LSP)

```
#include <iostream>
#include <string>
#include <vector>

//  Well-designed hierarchy following LSP

class Employee {
protected:
    std::string name;
    double baseSalary;

public:
    Employee(std::string n, double salary) : name(n), baseSalary(salary) {}

    // Core behaviors that ALL employees share
    virtual void doWork() {
        std::cout << name << " is doing work.\n";
    }

    virtual double calculatePay() const = 0;

    virtual ~Employee() = default;
};

class FullTimeEmployee : public Employee {
public:
    FullTimeEmployee(std::string n, double salary) : Employee(n, salary) {}

    double calculatePay() const override {
```

```
        return baseSalary / 12; // Monthly salary
    }

    void doWork() override {
        std::cout << name << " (Full-time) is working 8 hours.\n";
    }
};

class PartTimeEmployee : public Employee {
private:
    int hoursWorked;

public:
    PartTimeEmployee(std::string n, double hourlyRate, int hours)
        : Employee(n, hourlyRate * hours), hoursWorked(hours) {}

    double calculatePay() const override {
        return baseSalary; // Based on hours worked
    }

    void doWork() override {
        std::cout << name << " (Part-time) is working " << hoursWorked << " hours.\n";
    }
};

class ContractEmployee : public Employee {
private:
    int contractDays;
    int daysWorked;

public:
    ContractEmployee(std::string n, double dailyRate, int days)
```

```

        : Employee(n, dailyRate), contractDays(days), daysWorked(0) {}

double calculatePay() const override {
    return baseSalary * daysWorked; // Pay per completed days
}

void doWork() override {
    std::cout << name << " (Contract) is working on project.\n";
}

void completeDay() {
    daysWorked++;
}
};

// ✅ This function works correctly with ANY employee type
void processPayroll(const Employee& emp) {
    std::cout << "Processing payroll for " << emp.getName() << ": ";
    std::cout << "$" << emp.calculatePay() << "\n";
}

void assignTask(const Employee& emp) {
    emp.doWork(); // Polymorphism in action!
}

// ===== DEMONSTRATION =====
int main() {
    std::vector<Employee*> employees;

    employees.push_back(new FullTimeEmployee("Alice", 60000));
    employees.push_back(new PartTimeEmployee("Bob", 50, 20)); // $50/hr, 20 hrs
    ContractEmployee* contract = new ContractEmployee("Charlie", 500, 30);

```

```
contract->completeDay();
contract->completeDay();
contract->completeDay();
employees.push_back(contract);

std::cout << "=== Company Staff ===\n";
for (const auto& emp : employees) {
    processPayroll(*emp);
}

std::cout << "\n=== Work Assignments ===\n";
for (const auto& emp : employees) {
    assignTask(*emp);
}

std::cout << "\n=== After Contract Work ===\n";
processPayroll(*contract);

// Clean up
for (auto emp : employees) {
    delete emp;
}

return 0;
}
```

Output:

=== Company Staff ===

Processing payroll for Alice: \$5000

Processing payroll for Bob: \$1000

Processing payroll for Charlie: \$1500

=== Work Assignments ===

Alice (Full-time) is working 8 hours.

Bob (Part-time) is working 20 hours.

Charlie (Contract) is working on project.

=== After Contract Work ===

Processing payroll for Charlie: \$1500

Visual Representation

VIOLATION OF LSP

```

class Employee
  |
  |— work()      ← "I can work"
  |— takeBreak() ← "I can take breaks"
  |— getSalary() ← "I return salary"
  |
  ▼
class Intern (extends Employee)
  |
  |— getSalary() → returns 0.0 ← "My salary is zero"
  
```

Problem: If we treat Intern like Employee,
we expect a real salary but get nothing!

FOLLOWING LSP

```

class Employee (abstract)
  |
  |— doWork()      ← "All employees work"
  |— calculatePay() ← "All employees get paid somehow"
  |
  |—┬──────────┬──────────┬──────────┘
  |   ▼         ▼         ▼
FullTime    PartTime    Contract
- Monthly   - Hourly    - Per project
  
```

salary

wage

payment

- ✓ Any subclass can substitute Employee
- ✓ No broken expectations
- ✓ Polymorphism works correctly

LSP Checklist

For a subclass to properly substitute its parent:

- ✓ Parameter types must be compatible
- ✓ Return types must be compatible
- ✓ Preconditions must not be stronger (can't add stricter input requirements than parent)
- ✓ Postconditions must not be weaker (can't return less than parent promises)
- ✓ Exceptions thrown must be compatible (can't throw new types of exceptions)

4. I: Interface Segregation Principle (ISP)

Definition

"Clients should not be forced to depend on methods they do not use."

The Analogy

Think of a **Universal Remote** vs **Purpose-Specific Remotes**.

✗ BAD: Universal Remote (Forces you to use all features)

TV	AC	DVD	AUDIO	⚡
🔊	🌡️	📺	🎵	ON
VOL	TEMP	PLAY	BASS	POWER
You only watch TV, but you have all these buttons you never use!				

✓ GOOD: Separate Remotes (Each for specific purpose)

TV	AC	DVD
Remote	Remote	Remote
Power	Power	Power
Volume	Temp	Play
Channels	Mode	Stop
Input	Fan Speed	Menu

You only buy the remotes you need!

Bad Example (Violates ISP)

```
#include <iostream>
#include <string>
#include <vector>
#include <cmath>

// ✗ VIOLATION: One giant interface forcing all employees to implement
// everything, even if they don't need it

class IEmployee {
public:
    virtual void calculateSalary() = 0;
    virtual void generateReport() = 0;
    virtual void sendEmail() = 0;
    virtual void holdMeeting() = 0;
    virtual void manageProject() = 0;
    virtual void cleanWorkspace() = 0; // Everyone must implement this!
    virtual ~IEmployee() = default;
};

class Engineer : public IEmployee {
public:
    std::string name;

    Engineer(std::string n) : name(n) {}

    void calculateSalary() override {
        std::cout << "Calculating engineer's salary\n";
    }
}
```

```
void generateReport() override {
    std::cout << "Generating engineering report\n";
}

void sendEmail() override {
    std::cout << "Sending email\n";
}

void holdMeeting() override {
    std::cout << "Holding team meeting\n";
}

void manageProject() override {
    std::cout << "Managing project\n";
}

void cleanWorkspace() override {
    std::cout << "Cleaning workspace\n"; // Do engineers do this?
}

};

class Janitor : public IEmployee {
public:
    std::string name;

    Janitor(std::string n) : name(n) {}

    void calculateSalary() override {
        std::cout << "Calculating janitor's salary\n";
    }

    void generateReport() override {
```

```
// ❌ Janitors don't generate reports!
std::cout << "Janitor has nothing to report\n";
}

void sendEmail() override {
    // ❌ Janitors don't send emails!
    std::cout << "Janitor doesn't have email\n";
}

void holdMeeting() override {
    // ❌ Janitors don't hold meetings!
    std::cout << "Janitor doesn't hold meetings\n";
}

void manageProject() override {
    // ❌ Janitors don't manage projects!
    std::cout << "Janitor doesn't manage projects\n";
}

void cleanWorkspace() override {
    std::cout << "Janitor is cleaning the workspace\n";
}

};
```

Good Example (Follows ISP)

```
#include <iostream>
#include <string>
#include <vector>

//  GOOD: Segregated interfaces - each does one thing

// Interface for salary operations
class ISalaryCalculatable {
public:
    virtual void calculateSalary() = 0;
    virtual double getPayAmount() const = 0;
    virtual ~ISalaryCalculatable() = default;
};

// Interface for reporting operations
class IReportable {
public:
    virtual void generateReport() = 0;
    virtual std::string getReportData() const = 0;
    virtual ~IReportable() = default;
};

// Interface for communication operations
class ICommunicatable {
public:
    virtual void sendEmail() = 0;
    virtual void holdMeeting() = 0;
    virtual ~ICommunicatable() = default;
};
```

```
// Interface for project management
class IProjectManager {
public:
    virtual void manageProject() = 0;
    virtual void assignTask() = 0;
    virtual ~IProjectManager() = default;
};

// Interface for facility maintenance
class IFacilityManagement {
public:
    virtual void cleanWorkspace() = 0;
    virtual void maintainEquipment() = 0;
    virtual ~IFacilityManagement() = default;
};

//  Engineer implements what it needs
class Engineer : public ISalaryCalculatable, public IReportable,
                public ICommunicatable, public IProjectManager {
private:
    std::string name;
    double salary;

public:
    Engineer(std::string n, double s) : name(n), salary(s) {}

    void calculateSalary() override {
        std::cout << "Calculating engineer's salary: $" << salary << "\n";
    }

    double getPayAmount() const override {
```

```
        return salary;
    }

    void generateReport() override {
        std::cout << "Engineer " << name << " generating project report\n";
    }

    std::string getReportData() const override {
        return "Project progress report for " + name;
    }

    void sendEmail() override {
        std::cout << "Engineer " << name << " sending email\n";
    }

    void holdMeeting() override {
        std::cout << "Engineer " << name << " holding team meeting\n";
    }

    void manageProject() override {
        std::cout << "Engineer " << name << " managing project\n";
    }

    void assignTask() override {
        std::cout << "Engineer " << name << " assigning tasks\n";
    }
};

//  Janitor implements only what it needs
class Janitor : public ISalaryCalculatable, public IFacilityManagement {
private:
    std::string name;
```

```
double salary;

public:
    Janitor(std::string n, double s) : name(n), salary(s) {}

    void calculateSalary() override {
        std::cout << "Calculating janitor's salary: $" << salary << "\n";
    }

    double getPayAmount() const override {
        return salary;
    }

    void cleanWorkspace() override {
        std::cout << "Janitor " << name << " cleaning workspace\n";
    }

    void maintainEquipment() override {
        std::cout << "Janitor " << name << " maintaining equipment\n";
    }

    // ✅ No forced implementation of unrelated methods!
};

// ✅ Manager implements communication and project management
class Manager : public ISalaryCalculatable, public IReportable,
               public ICommunicatable, public IProjectManager {
private:
    std::string name;
    double salary;

public:
```

```
Manager(std::string n, double s) : name(n), salary(s) {}

void calculateSalary() override {
    std::cout << "Calculating manager's salary: $" << salary << "\n";
}

double getPayAmount() const override {
    return salary;
}

void generateReport() override {
    std::cout << "Manager " << name << " generating department report\n";
}

std::string getReportData() const override {
    return "Department performance report by " + name;
}

void sendEmail() override {
    std::cout << "Manager " << name << " sending company-wide email\n";
}

void holdMeeting() override {
    std::cout << "Manager " << name << " holding all-hands meeting\n";
}

void manageProject() override {
    std::cout << "Manager " << name << " overseeing multiple projects\n";
}

void assignTask() override {
    std::cout << "Manager " << name << " assigning tasks to team\n";
}
```



```
    }  
};  
  
// Helper function that only needs salary calculation  
void processPayroll(ISalaryCalculatable* employee) {  
    employee->calculateSalary();  
}  
  
// Helper function that only needs reporting  
void getReport(IReportable* employee) {  
    employee->generateReport();  
}  
  
// ===== DEMONSTRATION =====  
int main() {  
    std::cout << "=== Employee Operations by Role ===\n\n";  
  
    Engineer eng("Alice", 75000);  
    Manager mgr("Bob", 90000);  
    Janitor jan("Charlie", 35000);  
  
    std::cout << "--- Engineering Department ---\n";  
    processPayroll(&eng);  
    getReport(&eng);  
    eng.holdMeeting();  
    eng.manageProject();  
  
    std::cout << "\n--- Management ---\n";  
    processPayroll(&mgr);  
    getReport(&mgr);  
    mgr.sendEmail();  
    mgr.assignTask();  
}
```

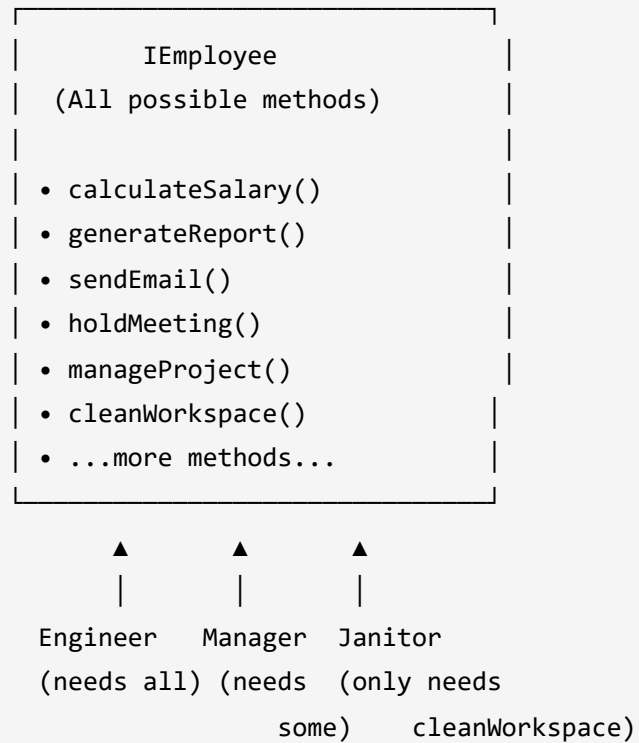
```
std::cout << "\n--- Facilities ---\n";  
processPayroll(&jan);  
jan.cleanWorkspace();  
jan.maintainEquipment();  
  
return 0;  
}
```

Output:

```
=== Employee Operations by Role ===  
  
--- Engineering Department ---  
Calculating engineer's salary: $75000  
Engineer Alice generating project report  
Engineer Alice holding team meeting  
Engineer Alice managing project  
  
--- Management ---  
Calculating manager's salary: $90000  
Manager Bob generating department report  
Manager Bob sending company-wide email  
Manager Bob assigning tasks to team  
  
--- Facilities ---  
Calculating janitor's salary: $35000  
Janitor Charlie cleaning workspace  
Janitor Charlie maintaining equipment
```

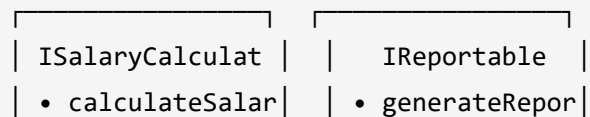
Visual Representation

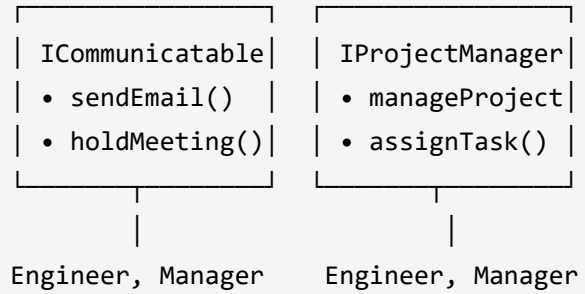
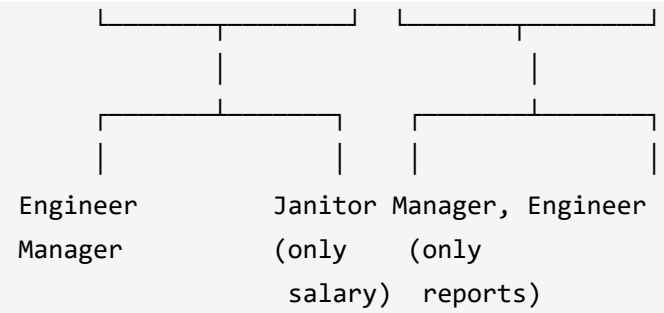
BAD DESIGN (Fat Interface)



✗ Every class forced to implement everything

GOOD DESIGN (Segregated Interfaces)





ISP vs SRP Connection

SRP: "One class should have one reason to change"

ISP: "Don't force classes to depend on methods they don't use"

Together:

	SRP focuses on WHAT the class does internally	
	ISP focuses on WHAT interfaces the class needs	
	SRP = Single Responsibility for classes	
	ISP = Interface Segregation for interfaces	
	Both lead to: SMALL, FOCUSED, COHESIVE components	

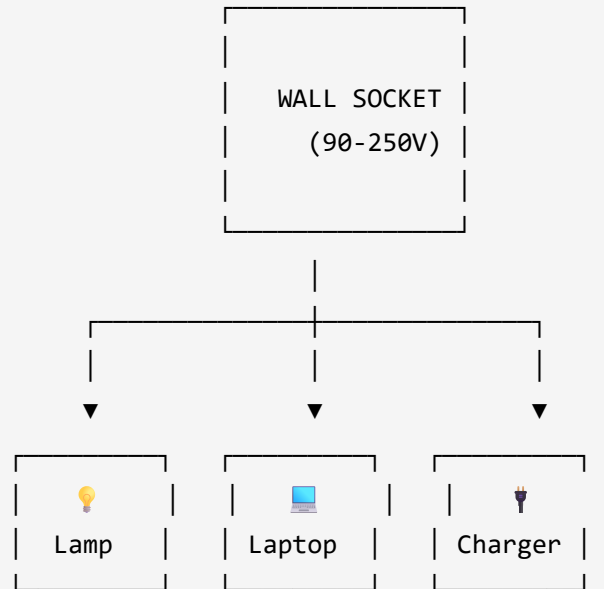
5. D: Dependency Inversion Principle (DIP)

Definition

"High-level modules should not depend on low-level modules. Both should depend on abstractions."

The Analogy

Think of a **Wall Socket** and **Electrical Devices**.



The WALL SOCKET doesn't depend on any specific device!
Any device that conforms to the electrical standard works.

High-level (Socket) $\leftarrow$ depends $\rightarrow$ Abstractions (Electricity Standard)
Low-level (Devices) $\leftarrow$ conforms $\rightarrow$

The socket doesn't care if you plug in a lamp, laptop, or phone charger. They all work because they follow the same interface (electricity).

Bad Example (Violates DIP)

```
#include <iostream>
#include <string>

// ✗ VIOLATION: High-level EmployeeManager depends on
// low-level implementation details

class MySQLDatabase {
public:
    void connect() {
        std::cout << "Connecting to MySQL database...\n";
    }

    void saveEmployee(std::string data) {
        std::cout << "Saving employee to MySQL: " << data << "\n";
    }
};

class FileStorage {
public:
    void saveEmployee(std::string data) {
        std::cout << "Saving employee to file: " << data << "\n";
    }
};

// ✗ High-level class depends directly on low-level implementation
class EmployeeManager {
private:
    MySQLDatabase database; // ✗ Tight coupling!
    // If we want to change to FileStorage, we must modify this class!
```

```
public:
    void saveEmployee(std::string name, double salary) {
        database.connect();
        std::string data = name + "|" + std::to_string(salary);
        database.saveEmployee(data);
    }
};

int main() {
    EmployeeManager mgr;
    mgr.saveEmployee("John", 50000);

    return 0;
}
```

Problem:

- `EmployeeManager` is tightly coupled to `MySQLDatabase`
- If we want to use `FileStorage` instead, we must modify `EmployeeManager`
- This violates Open/Closed Principle too!

Good Example (Follows DIP)

```
#include <iostream>
#include <string>
#include <vector>
#include <memory>

// ✅ GOOD: Abstractions (interfaces) first

// Abstract interface - defines contract for all storage
class IEmployeeRepository {
public:
    virtual void save(const std::string& name, double salary) = 0;
    virtual std::string getAll() const = 0;
    virtual ~IEmployeeRepository() = default;
};

// Low-level module: Implements the abstraction
class MySQLRepository : public IEmployeeRepository {
public:
    void save(const std::string& name, double salary) override {
        std::cout << "MySQL: Saving employee [" << name << ", $" << salary << "]\n";
    }

    std::string getAll() const override {
        return "Employees from MySQL database";
    }
};

// Low-level module: Another implementation
class FileRepository : public IEmployeeRepository {
```

```
public:
    void save(const std::string& name, double salary) override {
        std::cout << "File: Saving employee [" << name << ", $" << salary << "]\n";
    }

    std::string getAll() const override {
        return "Employees from file system";
    }
};

// Low-level module: Another implementation
class CloudRepository : public IEmployeeRepository {
public:
    void save(const std::string& name, double salary) override {
        std::cout << "Cloud: Saving employee [" << name << ", $" << salary << "]\n";
    }

    std::string getAll() const override {
        return "Employees from cloud storage";
    }
};

// ✅ High-level module depends on ABSTRACTION, not implementation
class EmployeeManager {
private:
    std::unique_ptr<IEmployeeRepository> repository; // Depends on abstraction!

public:
    // ✅ Inject the dependency - flexibility!
    EmployeeManager(std::unique_ptr<IEmployeeRepository> repo)
        : repository(std::move(repo)) {}
};
```

```
void addEmployee(const std::string& name, double salary) {
    repository->save(name, salary);
}

void printAllEmployees() {
    std::cout << "Report: " << repository->getAll() << "\n";
}

// ✅ Can change storage without modifying this class!
void changeStorage(std::unique_ptr<IEmployeeRepository> newRepo) {
    repository = std::move(newRepo);
}

};

// ✅ Factory pattern for easy switching
std::unique_ptr<IEmployeeRepository> createRepository(const std::string& type) {
    if (type == "mysql") {
        return std::make_unique<MySQLRepository>();
    } else if (type == "file") {
        return std::make_unique<FileRepository>();
    } else if (type == "cloud") {
        return std::make_unique<CloudRepository>();
    }
    return std::make_unique<FileRepository>(); // Default
}

// ===== DEMONSTRATION =====
int main() {
    std::cout << "=== DIP Demonstration ===\n\n";

    // Start with MySQL storage
    auto repo = createRepository("mysql");
```

```
EmployeeManager manager(std::move(repo));

std::cout << "--- Using MySQL ---\n";
manager.addEmployee("Alice", 75000);
manager.addEmployee("Bob", 65000);
manager.printAllEmployees();

// Switch to file storage - NO code changes to EmployeeManager!
std::cout << "\n--- Switching to File Storage ---\n";
manager.changeStorage(createRepository("file"));
manager.addEmployee("Charlie", 55000);
manager.printAllEmployees();

// Switch to cloud storage - still no changes!
std::cout << "\n--- Switching to Cloud Storage ---\n";
manager.changeStorage(createRepository("cloud"));
manager.addEmployee("Diana", 85000);
manager.printAllEmployees();

return 0;
}
```

Output:

=== DIP Demonstration ===

--- Using MySQL ---

MySQL: Saving employee [Alice, \$75000]

MySQL: Saving employee [Bob, \$65000]

Report: Employees from MySQL database

--- Switching to File Storage ---

File: Saving employee [Charlie, \$55000]

Report: Employees from file system

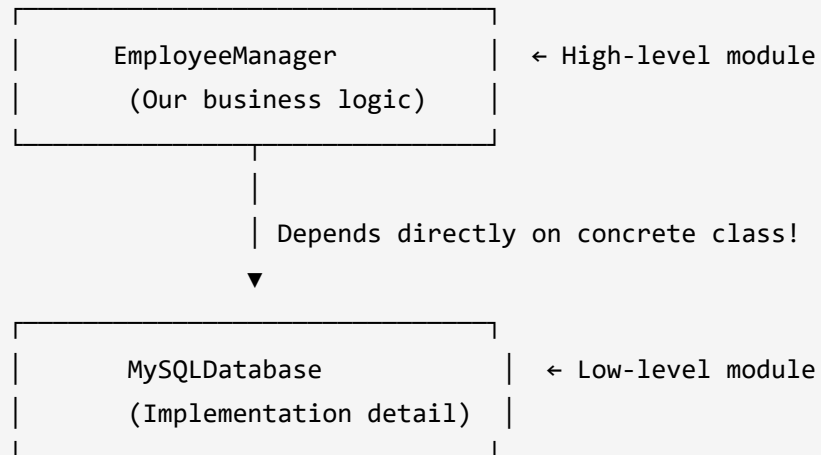
--- Switching to Cloud Storage ---

Cloud: Saving employee [Diana, \$85000]

Report: Employees from cloud storage

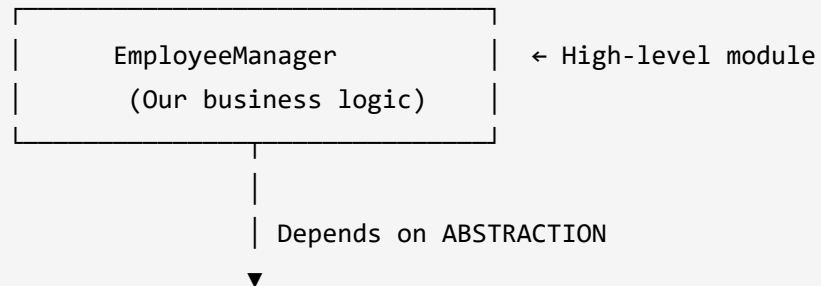
Visual Representation

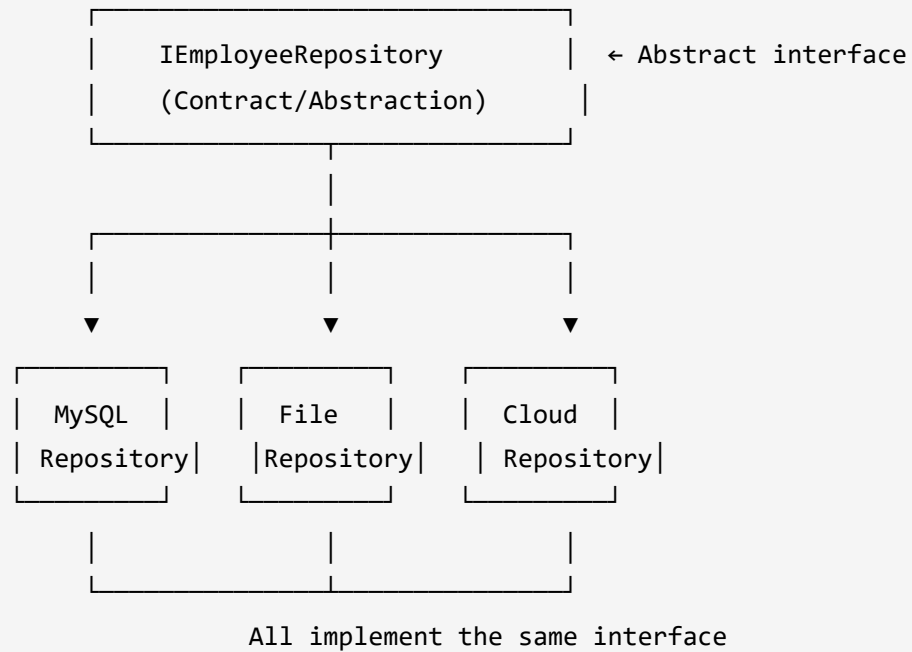
VIOLATION OF DIP



- ✗ If MySQL changes, EmployeeManager must change
- ✗ Can't switch to different storage easily
- ✗ Hard to test (can't mock MySQL easily)

FOLLOWING DIP





- ✓ EmployeeManager doesn't care about implementation
- ✓ Can switch storage at runtime
- ✓ Easy to test with mock repositories
- ✓ New storage types don't require code changes

Dependency Injection - Three Ways

```
// 1. Constructor Injection (Used in our example)
class EmployeeManager {
private:
    std::unique_ptr<IRepository> repository;
public:
    EmployeeManager(std::unique_ptr<IRepository> repo)
        : repository(std::move(repo)) {} // Injected via constructor
};

// 2. Setter Injection
class EmployeeManager {
private:
    IRepository* repository = nullptr;
public:
    void setRepository(IRepository* repo) { // Setter injection
        repository = repo;
    }
};

// 3. Interface Injection
class IRepositoryInjectable {
    virtual void injectRepository(IRepository* repo) = 0;
};
```


DIP Summary

DIP RULES
1. High-level modules should NOT know about low-level modules
2. Both should depend on ABSTRACTIONS
3. Abstractions should NOT depend on details
4. Details should depend on ABSTRACTIONS

BENEFITS:

- ✓ Flexibility - swap implementations easily
- ✓ Testability - mock dependencies for unit tests
- ✓ Maintainability - changes isolated to specific layers
- ✓ Extensibility - add new implementations without changing existing code

PART 2: DRY - Don't Repeat Yourself

Definition

"Every piece of knowledge must have a single, unambiguous, authoritative representation within a system."

The Analogy

Think of a **Reference Book** vs **Multiple Copies**.

✗ BAD: Multiple Copies Get Out of Sync

Library Book v1	Library Book v2	Library Book v3
Location: A-101	Location: A-102	Location: A-103
"Chapter 5..."	"Chapter 5..."	"Chapter 5..."
You update one copy. The other two are stale!	What about the other copies?	Now you have 3 different versions!

✓ GOOD: Single Source of Truth

MASTER COPY
Location: A-101
"Chapter 5..."
Everyone references THIS copy. When updated, all see the same truth. No inconsistencies!

Bad Example (Violates DRY)

```
#include <iostream>
#include <string>
#include <vector>

// ❌ VIOLATION: Duplicated logic everywhere!

class Employee {
private:
    std::string name;
    int id;
    double salary;

public:
    Employee(std::string n, int i, double s) : name(n), id(i), salary(s) {}

    // Same validation repeated in multiple places!

    // Validation logic #1 - in calculateBonus
    double calculateBonus() {
        if (salary > 0 && salary <= 50000) {
            return salary * 0.05;
        } else if (salary > 50000 && salary <= 100000) {
            return salary * 0.10;
        } else if (salary > 100000) {
            return salary * 0.15;
        }
        return 0;
    }
}
```

```
// Validation logic #2 - in calculateTax (DUPLICATED!)
double calculateTax() {
    if (salary > 0 && salary <= 50000) {
        return salary * 0.10; // 10% tax bracket
    } else if (salary > 50000 && salary <= 100000) {
        return salary * 0.20; // 20% tax bracket
    } else if (salary > 100000) {
        return salary * 0.30; // 30% tax bracket
    }
    return 0;
}

// Validation logic #3 - in determineInsurance (DUPLICATED AGAIN!)
double calculateInsuranceContribution() {
    if (salary > 0 && salary <= 50000) {
        return 1000; // Basic insurance
    } else if (salary > 50000 && salary <= 100000) {
        return 2000; // Standard insurance
    } else if (salary > 100000) {
        return 5000; // Premium insurance
    }
    return 0;
}

// What if salary bracket rules change? You have to update in 3 places!
// What if you make a mistake in one place but not others?
};

// Even worse - same code across different classes!
class Contractor {
private:
    std::string name;
```

```
    int id;
    double hourlyRate;
    int hoursWorked;

public:
    Contractor(std::string n, int i, double rate, int hours)
        : name(n), id(i), hourlyRate(rate), hoursWorked(hours) {}

    // Same salary bracket logic copied here too!
    double calculateMonthlyPay() {
        double total = hourlyRate * hoursWorked;
        if (total > 0 && total <= 50000) {
            return total * 0.90; // After 10% deduction
        } else if (total > 50000 && total <= 100000) {
            return total * 0.85; // After 15% deduction
        } else if (total > 100000) {
            return total * 0.80; // After 20% deduction
        }
        return 0;
    }
};

// This code has THREE instances of the same salary bracket logic!
// If rules change, all three must be updated.
// Miss one? You've introduced a bug.
```

Good Example (Follows DRY)

```
#include <iostream>
#include <string>
#include <vector>
#include <functional>

// ✅ GOOD: Single source of truth for salary bracket logic

// ===== SALARY BRACKET ENUM =====
enum class SalaryBracket {
    LOW,          // <= $50,000
    MEDIUM,      // $50,001 - $100,000
    HIGH          // > $100,000
};

// ===== SALARY CALCULATOR CLASS =====
class SalaryCalculator {
private:
    // Constants defined once - easy to maintain
    static constexpr double LOW_THRESHOLD = 50000;
    static constexpr double MEDIUM_THRESHOLD = 100000;

public:
    // Single method to determine bracket
    static SalaryBracket getBracket(double salary) {
        if (salary <= LOW_THRESHOLD) {
            return SalaryBracket::LOW;
        } else if (salary <= MEDIUM_THRESHOLD) {
            return SalaryBracket::MEDIUM;
        }
    }
}
```

```
        return SalaryBracket::HIGH;
    }

    // All calculations use the same bracket logic - DRY!
    static double calculateBonus(double salary) {
        switch (getBracket(salary)) {
            case SalaryBracket::LOW:
                return salary * 0.05;
            case SalaryBracket::MEDIUM:
                return salary * 0.10;
            case SalaryBracket::HIGH:
                return salary * 0.15;
        }
        return 0;
    }

    static double calculateTax(double salary) {
        switch (getBracket(salary)) {
            case SalaryBracket::LOW:
                return salary * 0.10;
            case SalaryBracket::MEDIUM:
                return salary * 0.20;
            case SalaryBracket::HIGH:
                return salary * 0.30;
        }
        return 0;
    }

    static double calculateInsuranceContribution(double salary) {
        switch (getBracket(salary)) {
            case SalaryBracket::LOW:
                return 1000;
        }
    }
}
```

```
        case SalaryBracket::MEDIUM:
            return 2000;
        case SalaryBracket::HIGH:
            return 5000;
    }
    return 0;
}

// If rules change, only this class needs updating!
// And it affects ALL calculations uniformly.
};

// ===== EMPLOYEE CLASS - Uses the centralized calculator =====
class Employee {
private:
    std::string name;
    int id;
    double salary;

public:
    Employee(std::string n, int i, double s) : name(n), id(i), salary(s) {}

    double getSalary() const { return salary; }

    // All calculations delegate to SalaryCalculator - no duplication!
    double calculateBonus() {
        return SalaryCalculator::calculateBonus(salary);
    }

    double calculateTax() {
        return SalaryCalculator::calculateTax(salary);
    }
}
```



```

double calculateInsurance() {
    return SalaryCalculator::calculateInsuranceContribution(salary);
}

void printCompensationReport() {
    std::cout << "=== Employee: " << name << " ===\n";
    std::cout << "Salary: $" << salary << "\n";
    std::cout << "Bonus: $" << calculateBonus() << "\n";
    std::cout << "Tax: $" << calculateTax() << "\n";
    std::cout << "Insurance: $" << calculateInsurance() << "\n";

    SalaryBracket bracket = SalaryCalculator::getBracket(salary);
    std::cout << "Bracket: ";
    switch (bracket) {
        case SalaryBracket::LOW: std::cout << "LOW"; break;
        case SalaryBracket::MEDIUM: std::cout << "MEDIUM"; break;
        case SalaryBracket::HIGH: std::cout << "HIGH"; break;
    }
    std::cout << "\n===== \n";
}

};

// ===== CONTRACTOR CLASS - Also uses the same calculator! =====
class Contractor {
private:
    std::string name;
    int id;
    double hourlyRate;
    int hoursPerWeek;

public:

```

```
Contractor(std::string n, int i, double rate, int hours)
    : name(n), id(i), hourlyRate(rate), hoursPerWeek(hours) {}

double getWeeklyEarnings() const {
    return hourlyRate * hoursPerWeek;
}

double getAnnualEarnings() const {
    return getWeeklyEarnings() * 52; // Assuming 52 weeks
}

// Uses the SAME salary calculator - no duplication!
double calculateAnnualDeduction() {
    return SalaryCalculator::calculateTax(getAnnualEarnings());
}

void printCompensationReport() {
    std::cout << "=== Contractor: " << name << " ===\n";
    std::cout << "Annual Earnings: $" << getAnnualEarnings() << "\n";
    std::cout << "Annual Deduction: $" << calculateAnnualDeduction() << "\n";

    SalaryBracket bracket = SalaryCalculator::getBracket(getAnnualEarnings());
    std::cout << "Bracket: ";
    switch (bracket) {
        case SalaryBracket::LOW: std::cout << "LOW"; break;
        case SalaryBracket::MEDIUM: std::cout << "MEDIUM"; break;
        case SalaryBracket::HIGH: std::cout << "HIGH"; break;
    }
    std::cout << "\n===== \n";
}

};
```

```
// ===== DEMONSTRATION =====  
  
int main() {  
    std::cout << "=== DRY Principle Demonstration ===\n\n";  
  
    Employee emp("Alice", 101, 75000);  
    emp.printCompensationReport();  
  
    Contractor con("Bob", 201, 45, 40); // $45/hr, 40 hrs/week  
    con.printCompensationReport();  
  
    // If tax rules change, update SalaryCalculator in ONE place  
    // Both Employee and Contractor automatically get the fix!  
  
    std::cout << "Notice: Both use the same SalaryCalculator.\n";  
    std::cout << "Change the bracket rules once, affects everyone!\n";  
  
    return 0;  
}
```

Output:

=== DRY Principle Demonstration ===

=== Employee: Alice ===

Salary: \$75000

Bonus: \$7500

Tax: \$15000

Insurance: \$2000

Bracket: MEDIUM

=====

=== Contractor: Bob ===

Annual Earnings: \$93600

Annual Deduction: \$18720

Bracket: MEDIUM

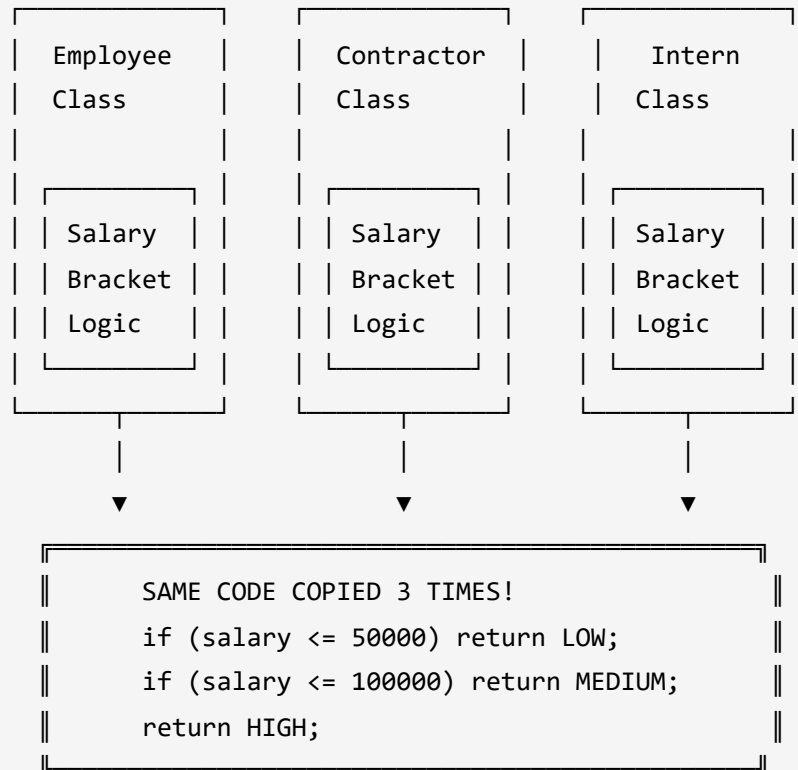
=====

Notice: Both use the same SalaryCalculator.

Change the bracket rules once, affects everyone!

Visual Representation

VIOLATION OF DRY

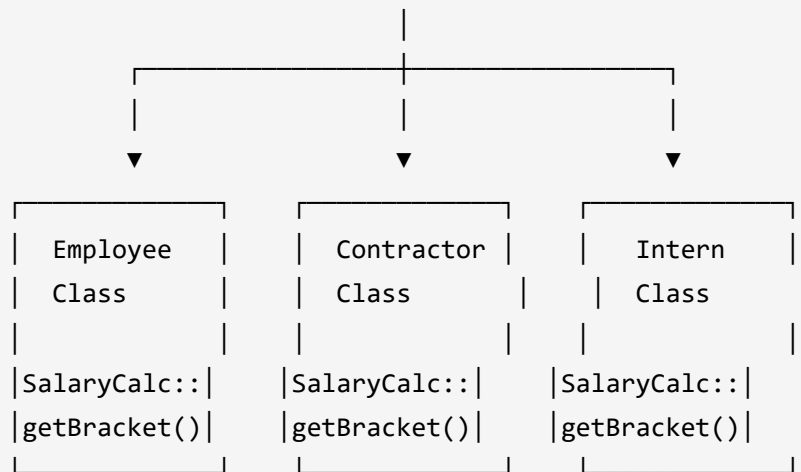


- ✗ Change in one place might not be applied to others
- ✗ Inconsistency bugs
- ✗ Harder to maintain

FOLLOWING DRY

```
SalaryCalculator
(Single Source of Truth)

static SalaryBracket getBracket(double salary) {
    if (salary <= 50000) return LOW;
    if (salary <= 100000) return MEDIUM;
    return HIGH;
}
```



- ✓ One change updates all
- ✓ Consistency guaranteed
- ✓ Easy to maintain

DRY vs WET

DRY vs WET

DRY = "Don't Repeat Yourself"

WET = "Write Every Time" or "Waste Everyone's Time"

DRY Code:

```
calculateTax()  
├─ Uses SalaryCalculator  
└─ One place to maintain
```

WET Code:

```
calculateTax()      ← copy 1  
calculateBonus()    ← copy 2  
calculateDeduction()← copy 3  
// Same logic in each function!
```

PART 3: KISS - Keep It Simple, Stupid

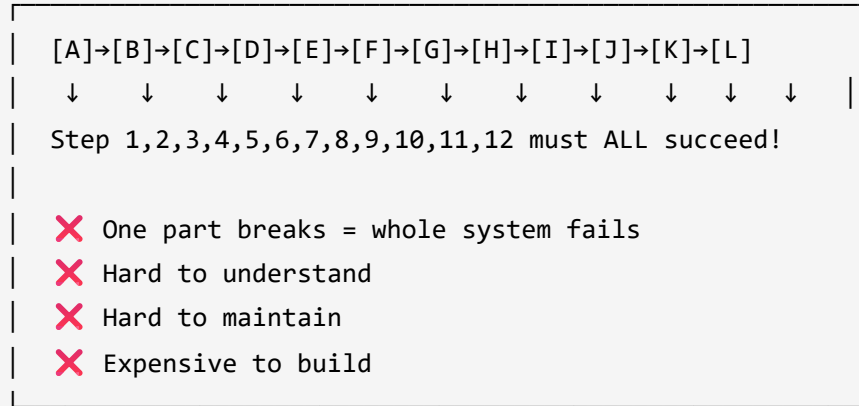
Definition

"Most systems work best if they are kept simple. Simplicity should be a key goal, and unnecessary complexity should be avoided."

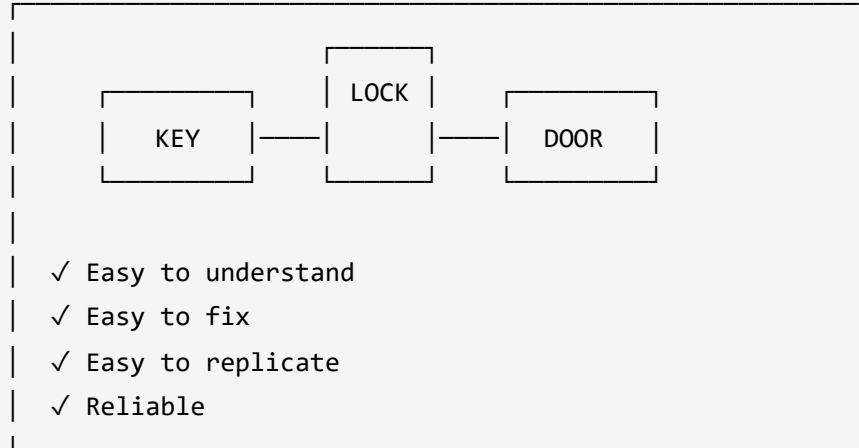
The Analogy

Think of a **Simple Lock** vs **Complicated Lock**.

✗ OVERENGINEERED: 50 moving parts, 12-step process to unlock



✓ SIMPLE: One key, one lock, works every time



Bad Example (Violates KISS)

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>

// ✗ VIOLATION: Overcomplicated design for a simple task

class Employee {
private:
    std::string name;
    int id;
    double salary;
    std::string department;
    std::string email;
    std::string phone;
    std::string address;
    std::string hireDate;
    std::string position;
    int performanceRating;
    bool isActive;
    double bonusPercentage;
    int sickDaysLeft;
    int vacationDaysLeft;
    std::string managerId;
    std::vector<std::string> skills;
    std::vector<std::string> certifications;
    // ... and many more fields that might not always be needed!

public:
```

```
Employee(std::string n, int i, double s) : name(n), id(i), salary(s) {}

// Constructor with 47 parameters for every possible scenario
Employee(std::string n, int i, double s, std::string dept, std::string email,
        std::string phone, std::string address, std::string hireDate,
        std::string position, int perfRating, bool active, double bonus,
        int sickDays, int vacDays, std::string mgrId,
        std::vector<std::string> skillsList, std::vector<std::string> certs,
        std::string someOtherField, std::string anotherField)
: name(n), id(i), salary(s), department(dept), email(email),
  phone(phone), address(address), hireDate(hireDate), position(position),
  performanceRating(perfRating), isActive(active), bonusPercentage(bonus),
  sickDaysLeft(sickDays), vacationDaysLeft(vacDays), managerId(mgrId),
  skills(skillsList), certifications(certs) {}

// Method that does 15 different things
void processEmployeeData() {
    // Validate name
    // Validate email format
    // Check phone number format
    // Verify address
    // Calculate years of service
    // Determine bonus eligibility
    // Check certification status
    // Calculate performance score
    // Update vacation balance
    // Send notification to manager
    // Log activity
    // Update database
    // Generate report
    // Send welcome email
    // ... 50 more operations
}
```

```
    }  
};  
  
// Class with nested classes inside classes inside classes  
class EmployeeManagementSystem {  
private:  
    class InternalDataProcessor {  
private:  
        class DeepNestedHelper {  
private:  
            class EvenDeeperClass {  
public:  
                void doSomething() {  
                    // Nested 4 levels deep just to calculate 2+2  
                }  
            };  
        };  
    };  
};  
  
public:  
    // Overcomplicated factory pattern when simple constructor would work  
    class EmployeeBuilder {  
private:  
        Employee* tempEmployee;  
  
public:  
        EmployeeBuilder() { tempEmployee = nullptr; }  
  
        EmployeeBuilder* create() {  
            tempEmployee = new Employee("", 0, 0);  
            return this;  
        }  
    }  
};
```

```
EmployeeBuilder* withName(std::string name) {  
    // 50 lines of validation  
    return this;  
}  
  
EmployeeBuilder* withId(int id) {  
    // 50 more lines of validation  
    return this;  
}  
  
EmployeeBuilder* build() {  
    return this;  
}  
};  
};
```

Good Example (Follows KISS)

```
#include <iostream>
#include <string>
#include <vector>

// ✅ GOOD: Simple, focused design

class Employee {
private:
    std::string name;
    int id;
    double salary;

public:
    // Simple constructor - only what we need right now
    Employee(std::string n, int i, double s) : name(n), id(i), salary(s) {}

    // Simple getters - nothing fancy
    std::string getName() const { return name; }
    int getId() const { return id; }
    double getSalary() const { return salary; }

    void setSalary(double s) { salary = s; } // Simple setter
};

// Separate classes for separate concerns - but each is SIMPLE
class SalaryCalculator {
public:
    static double calculateBonus(double salary, double percentage) {
        return salary * (percentage / 100.0);
    }
};
```

```
    }

    static double calculateTax(double salary, double rate) {
        return salary * (rate / 100.0);
    }
};

class EmployeeValidator {
public:
    static bool isValidSalary(double salary) {
        return salary >= 0 && salary < 1000000;
    }

    static bool isValidName(const std::string& name) {
        return !name.empty() && name.length() < 100;
    }
};

class EmployeePrinter {
public:
    static void printBasic(const Employee& emp) {
        std::cout << "Name: " << emp.getName()
                  << ", ID: " << emp.getId()
                  << ", Salary: $" << emp.getSalary() << "\n";
    }
};

// Simple usage - easy to understand, easy to use
int main() {
    Employee emp("Alice", 101, 50000);

    std::cout << "=== KISS Demonstration ===\n\n";
```

```
EmployeePrinter::printBasic(emp);

double bonus = SalaryCalculator::calculateBonus(emp.getSalary(), 10);
std::cout << "Bonus (10%): $" << bonus << "\n";

double tax = SalaryCalculator::calculateTax(emp.getSalary(), 20);
std::cout << "Tax (20%): $" << tax << "\n";

// Each class does ONE thing simply
// No overengineering
// Easy to test
// Easy to maintain

return 0;
}
```

Output:

=== KISS Demonstration ===

Name: Alice, ID: 101, Salary: \$50000

Bonus (10%): \$5000

Tax (20%): \$10000

More KISS Examples

```
// ❌ OVERCOMPLEX: Nested ternaries and clever tricks
int getEmployeeLevel(Employee& emp) {
    return emp.getSalary() > 100000 ?
        (emp.getYearsOfService() > 5 ? 5 : 4) :
        (emp.getSalary() > 50000 ?
            (emp.getPerformanceRating() > 4 ? 3 : 2) : 1);
}
```

```
// ✅ SIMPLE: Clear, readable code
int getEmployeeLevel(Employee& emp) {
    if (emp.getSalary() > 100000) {
        return emp.getYearsOfService() > 5 ? 5 : 4;
    } else if (emp.getSalary() > 50000) {
        return emp.getPerformanceRating() > 4 ? 3 : 2;
    }
    return 1;
}
```

```
// ❌ OVERCOMPLEX: Singleton with double-check locking
class EmployeeDatabase {
private:
    static EmployeeDatabase* instance;
    static std::mutex mutex;

public:
    static EmployeeDatabase* getInstance() {
        if (instance == nullptr) {
            std::lock_guard<std::mutex> lock(mutex);
            if (instance == nullptr) {
```

```
        instance = new EmployeeDatabase();
    }
}
return instance;
}
};

// ✅ SIMPLE: Dependency injection (or even just a regular class)
class EmployeeDatabase {
private:
    // Just use it directly - no singleton needed for most cases
    std::vector<Employee> employees;

public:
    void addEmployee(const Employee& emp) {
        employees.push_back(emp);
    }
};
```

KISS Decision Guide

SHOULD I USE KISS?

Ask yourself these questions:

1. Can a new developer understand this in 5 minutes?

→ If no, it's too complex

2. Will I need all these features today?

→ If no, don't add them yet (YAGNI)

3. Is there a simpler way to solve this?

→ Always look for the simpler solution first

4. Am I adding complexity for "just in case"?

→ Don't solve problems you don't have yet

PART 4: Practical Application of DRY and KISS

Combining Both Principles

Example: Employee Management System

```
#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <sstream>

// =====
// PRACTICAL APPLICATION: Combining DRY and KISS
// =====

// ===== SINGLE SOURCE OF TRUTH (DRY) =====

class ValidationRules {
public:
    static constexpr int MIN_NAME_LENGTH = 2;
    static constexpr int MAX_NAME_LENGTH = 50;
    static constexpr double MIN_SALARY = 0;
    static constexpr double MAX_SALARY = 1000000;
    static constexpr int MIN_ID = 1;
    static constexpr int MAX_ID = 999999;

    static bool isValidName(const std::string& name) {
```

```

    if (name.length() < MIN_NAME_LENGTH || name.length() > MAX_NAME_LENGTH) {
        return false;
    }
    // Check for valid characters
    for (char c : name) {
        if (!std::isalpha(c) && c != ' ' && c != '-') {
            return false;
        }
    }
    return true;
}

static bool isValidSalary(double salary) {
    return salary >= MIN_SALARY && salary <= MAX_SALARY;
}

static bool isValidId(int id) {
    return id >= MIN_ID && id <= MAX_ID;
}

static std::string getValidationError(const std::string& field, double value) {
    std::ostringstream oss;
    oss << field << " must be between " << MIN_SALARY << " and " << MAX_SALARY;
    return oss.str();
}
};

// ===== SIMPLE EMPLOYEE CLASS (KISS) =====

class Employee {
private:
    std::string name;

```

```
    int id;
    double salary;

public:
    Employee(std::string n, int i, double s) : name(n), id(i), salary(s) {
        // Single place for validation (DRY)
        if (!ValidationRules::isValidName(n)) {
            throw std::invalid_argument("Invalid name");
        }
        if (!ValidationRules::isValidId(i)) {
            throw std::invalid_argument("Invalid ID");
        }
        if (!ValidationRules::isValidSalary(s)) {
            throw std::invalid_argument("Invalid salary");
        }
    }

    // Simple getters
    std::string getName() const { return name; }
    int getId() const { return id; }
    double getSalary() const { return salary; }

    void setSalary(double s) {
        if (!ValidationRules::isValidSalary(s)) {
            throw std::invalid_argument("Invalid salary");
        }
        salary = s;
    }
};

// ===== SEPARATE CONCERNS (KISS + SRP) =====
```

```
// Format strings in ONE place (DRY)
class Formatter {
public:
    static std::string formatCurrency(double amount) {
        std::ostringstream oss;
        oss << "$" << std::fixed << std::setprecision(2) << amount;
        return oss.str();
    }

    static std::string formatEmployeeLine(const Employee& emp) {
        std::ostringstream oss;
        oss << emp.getName() << " | "
            << emp.getId() << " | "
            << Formatter::formatCurrency(emp.getSalary());
        return oss.str();
    }
};

// Calculate things in ONE place (DRY)
class PayrollCalculator {
public:
    static double calculateNetSalary(double grossSalary) {
        double tax = calculateTax(grossSalary);
        double insurance = calculateInsurance(grossSalary);
        return grossSalary - tax - insurance;
    }

    static double calculateTax(double salary) {
        // Simple tax calculation - one place (DRY)
        if (salary <= 50000) return salary * 0.10;
        if (salary <= 100000) return salary * 0.20;
        return salary * 0.30;
    }
};
```

```
}

static double calculateInsurance(double salary) {
    // Single insurance calculation (DRY)
    if (salary <= 50000) return 1000;
    if (salary <= 100000) return 2000;
    return 5000;
}

static double calculateBonus(double salary, double percentage) {
    return salary * (percentage / 100.0);
}
};

// ===== REPORT GENERATION (DRY + KISS) =====

class EmployeeReport {
private:
    std::vector<Employee> employees;

public:
    void addEmployee(const Employee& emp) {
        employees.push_back(emp);
    }

    std::string generateSummary() {
        std::ostringstream oss;
        oss << "=== EMPLOYEE SUMMARY REPORT ===\n";
        oss << "Total Employees: " << employees.size() << "\n";

        double totalSalary = 0;
        for (const auto& emp : employees) {
```



```
        totalSalary += emp.getSalary();
    }
    oss << "Total Salary Budget: " << Formatter::formatCurrency(totalSalary) << "\n";
    oss << "Average Salary: " << Formatter::formatCurrency(
        employees.empty() ? 0 : totalSalary / employees.size()) << "\n";

    return oss.str();
}

std::string generateDetailedReport() {
    std::ostringstream oss;
    oss << "=== DETAILED EMPLOYEE REPORT ===\n";
    for (const auto& emp : employees) {
        double netSalary = PayrollCalculator::calculateNetSalary(emp.getSalary());
        double bonus = PayrollCalculator::calculateBonus(emp.getSalary(), 10);
        double tax = PayrollCalculator::calculateTax(emp.getSalary());

        oss << "\n" << Formatter::formatEmployeeLine(emp) << "\n";
        oss << "  Gross: " << Formatter::formatCurrency(emp.getSalary()) << "\n";
        oss << "  Tax: " << Formatter::formatCurrency(tax) << "\n";
        oss << "  Insurance: " << Formatter::formatCurrency(
            PayrollCalculator::calculateInsurance(emp.getSalary())) << "\n";
        oss << "  Net: " << Formatter::formatCurrency(netSalary) << "\n";
        oss << "  Bonus: " << Formatter::formatCurrency(bonus) << "\n";
    }
    return oss.str();
}

std::string generateCSV() {
    std::ostringstream oss;
    oss << "Name,ID,Salary,Net,Tax,Bonus\n";
    for (const auto& emp : employees) {
```

```
double net = PayrollCalculator::calculateNetSalary(emp.getSalary());
double tax = PayrollCalculator::calculateTax(emp.getSalary());
double bonus = PayrollCalculator::calculateBonus(emp.getSalary(), 10);

oss << emp.getName() << ", "
    << emp.getId() << ", "
    << emp.getSalary() << ", "
    << net << ", "
    << tax << ", "
    << bonus << "\n";
}
return oss.str();
}
};

// ===== DEMONSTRATION =====

int main() {
    std::cout << "=== DRY + KISS Practical Application ===\n\n";

    EmployeeReport report;

    // Create employees - validation happens in one place
    try {
        report.addEmployee(Employee("Alice Smith", 101, 75000));
        report.addEmployee(Employee("Bob Johnson", 102, 55000));
        report.addEmployee(Employee("Carol Williams", 103, 95000));
        report.addEmployee(Employee("David Brown", 104, 45000));
    } catch (const std::invalid_argument& e) {
        std::cout << "Error: " << e.what() << "\n";
    }
}
```

```
// Generate different reports using the same data (DRY)
std::cout << report.generateSummary();
std::cout << "\n" << report.generateDetailedReport();

std::cout << "\n=== CSV Export ===\n";
std::cout << report.generateCSV();

// The key benefits:
// 1. Validation rules defined once (DRY)
// 2. Calculations in one place (DRY)
// 3. Easy to understand classes (KISS)
// 4. Easy to add new report formats (OCP)
// 5. Easy to test each component (SRP)

return 0;
}
```

Output:

=== DRY + KISS Practical Application ===

=== EMPLOYEE SUMMARY REPORT ===

Total Employees: 4

Total Salary Budget: \$270000.00

Average Salary: \$67500.00

=== DETAILED EMPLOYEE REPORT ===

Alice Smith | 101 | \$75000.00

Gross: \$75000.00

Tax: \$15000.00

Insurance: \$2000.00

Net: \$58000.00

Bonus: \$7500.00

Bob Johnson | 102 | \$55000.00

Gross: \$55000.00

Tax: \$11000.00

Insurance: \$2000.00

Net: \$42000.00

Bonus: \$5500.00

Carol Williams | 103 | \$95000.00

Gross: \$95000.00

Tax: \$19000.00

Insurance: \$2000.00

Net: \$74000.00

Bonus: \$9500.00

David Brown | 104 | \$45000.00

Gross: \$45000.00

Tax: \$4500.00

Insurance: \$1000.00

Net: \$39500.00

Bonus: \$4500.00

=== CSV Export ===

Name,ID,Salary,Net,Tax,Bonus

Alice Smith,101,75000,58000,15000,7500

Bob Johnson,102,55000,42000,11000,5500

Carol Williams,103,95000,74000,19000,9500

David Brown,104,45000,39500,4500,4500

Design Decisions Summary

COMBINING DRY + KISS = GOOD SOFTWARE DESIGN

DRY ("Don't Repeat Yourself")

- ✓ Single source of truth for validation
- ✓ Calculations defined once and reused
- ✓ Consistent formatting across entire codebase
- ✓ Easy to update - change in one place affects all

KISS ("Keep It Simple, Stupid")

- ✓ Employee class has only essential fields
- ✓ Each helper class has one responsibility
- ✓ Clear, readable method names
- ✓ Easy for new developers to understand

RESULT: Maintainable, Testable, Extensible Code

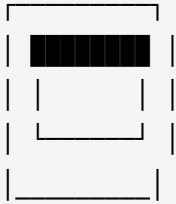
PART 5: Designing for Scalability and Maintainability

Introduction

Scalability and maintainability are two pillars of enterprise software design. While functionality gets your software working, scalability and maintainability determine how well it grows and survives over time.

Think of a **Skyscraper** vs a **Small House**:

✗ SMALL HOUSE - Hard to expand

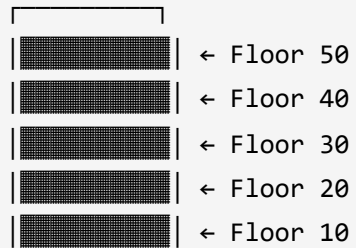


Add another floor?

Need to rebuild from scratch!

Foundation too weak.

✓ SKYSCRAPER - Built to scale



Designed for growth from day one!

5a. Scalability in Design

Definition

Scalability is the ability of a system to handle increased load by adding resources (vertical) or distributing load across multiple resources (horizontal) without major redesign.

The Analogy

Think of a **Single Lane Road** vs a **Highway**.

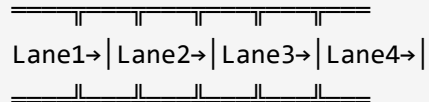
✖ SINGLE LANE ROAD - Not scalable



Problems:

- Only one car at a time
- Add more cars = traffic jam
- Can't handle rush hour
- Need to rebuild for more capacity

✔ HIGHWAY - Horizontally scalable



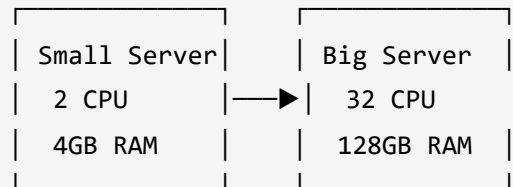
Features:

- Multiple lanes handle more traffic
- Add more lanes without rebuilding
- Different lanes for different needs (carpool, express)
- Can handle rush hour!

Types of Scalability

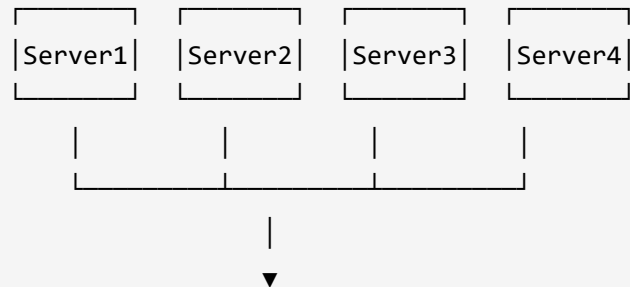
SCALABILITY TYPES

VERTICAL SCALING (Scale Up)



- ✓ Easier to implement
- X Hardware limits
- X Single point of failure

HORIZONTAL SCALING (Scale Out)



Load Balancer

- ✓ Add more servers easily
- ✓ No single point of failure
- ✓ More cost-effective at scale
- X More complex to manage

Bad Example (Not Scalable)

```
#include <iostream>
#include <string>
#include <vector>

// ❌ VIOLATION: Not designed for scalability

class EmployeeDatabase {
private:
    std::vector<Employee> employees; // All data in memory!

public:
    void addEmployee(const Employee& emp) {
        employees.push_back(emp); // Gets slower as data grows
    }

    Employee* findById(int id) {
        // O(n) linear search - gets slower with more data
        for (auto& emp : employees) {
            if (emp.getId() == id) {
                return &emp;
            }
        }
        return nullptr;
    }

    std::vector<Employee> findByName(const std::string& name) {
        std::vector<Employee> results;
        // O(n) scan through all employees
        for (auto& emp : employees) {
```

```
        if (emp.getName().find(name) != std::string::npos) {
            results.push_back(emp);
        }
    }
    return results;
}

size_t getCount() const { return employees.size(); }
// What happens when we have 1 million employees?
// - Memory runs out
// - Search becomes too slow
// - Can't distribute across servers
};

// Single-threaded server
class EmployeeServer {
private:
    EmployeeDatabase db;

public:
    void handleRequest(const std::string& request) {
        // Only one request at a time!
        std::cout << "Processing: " << request << "\n";
        // One request blocks all others
    }
};

// Problems with this design:
// 1. All data in memory - memory limit reached quickly
// 2. Linear search - O(n) becomes too slow
```

```
// 3. Single-threaded - can't handle concurrent users  
// 4. Can't distribute - all on one machine
```

Good Example (Scalable Design)

```
#include <iostream>
#include <string>
#include <vector>
#include <unordered_map>
#include <memory>
#include <thread>
#include <queue>
#include <mutex>

// ✅ GOOD: Scalable design with multiple patterns

// ===== 1. INDEXING FOR FASTER LOOKUPS (O(1) vs O(n)) =====

class EmployeeIndex {
private:
    // Hash map for O(1) lookup by ID
    std::unordered_map<int, Employee> idIndex;
    // Hash map for O(1) lookup by department
    std::unordered_map<std::string, std::vector<Employee>> departmentIndex;
    // Could add more indexes as needed

public:
    void addEmployee(const Employee& emp) {
        idIndex[emp.getId()] = emp;
        departmentIndex[emp.getDepartment()].push_back(emp);
    }

    // O(1) lookup instead of O(n) linear search!
    Employee* findById(int id) {
```



```
    auto it = idIndex.find(id);
    if (it != idIndex.end()) {
        return &it->second;
    }
    return nullptr;
}

// O(1) lookup by department
std::vector<Employee> findByDepartment(const std::string& dept) {
    auto it = departmentIndex.find(dept);
    if (it != departmentIndex.end()) {
        return it->second;
    }
    return {};
}
};

// ===== 2. PAGINATION FOR LARGE DATASETS =====

class EmployeeRepository {
private:
    std::vector<Employee> employees;
    static constexpr size_t PAGE_SIZE = 100;

public:
    void addEmployee(const Employee& emp) {
        employees.push_back(emp);
    }

    // Return only a PAGE at a time - scalable!
    std::vector<Employee> getPage(size_t pageNumber) {
        size_t start = pageNumber * PAGE_SIZE;
```

```

    if (start >= employees.size()) {
        return {};
    }

    size_t end = std::min(start + PAGE_SIZE, employees.size());
    return std::vector<Employee>(employees.begin() + start,
                                employees.begin() + end);
}

size_t getTotalPages() const {
    return (employees.size() + PAGE_SIZE - 1) / PAGE_SIZE;
}

size_t getTotalCount() const { return employees.size(); }
};

// ===== 3. THREAD POOL FOR CONCURRENT REQUESTS =====

class ThreadPool {
private:
    std::vector<std::thread> workers;
    std::queue<std::function<void()>> tasks;
    std::mutex queueMutex;
    bool stop = false;

public:
    ThreadPool(size_t numThreads) {
        for (size_t i = 0; i < numThreads; ++i) {
            workers.emplace_back([this] {
                while (true) {
                    std::function<void()> task;
                    {

```

```

        std::lock_guard<std::mutex> lock(queueMutex);
        if (stop || tasks.empty()) {
            return;
        }
        task = std::move(tasks.front());
        tasks.pop();
    }
    task();
}
});
}
}

```

```

template<typename F>
void enqueue(F&& f) {
    {
        std::lock_guard<std::mutex> lock(queueMutex);
        tasks.emplace(std::forward<F>(f));
    }
}

```

```

~ThreadPool() {
    stop = true;
    for (auto& worker : workers) {
        worker.join();
    }
}
};

```

```
// ===== 4. INTERFACE FOR MULTIPLE STORAGE BACKENDS =====
```

```
// Abstract interface - enables horizontal scaling with different storage
```

```
class IEmployeeStorage {
public:
    virtual void save(const Employee& emp) = 0;
    virtual Employee* find(int id) = 0;
    virtual std::vector<Employee> findAll() = 0;
    virtual ~IEmployeeStorage() = default;
};

// In-memory storage (for small scale)
class InMemoryEmployeeStorage : public IEmployeeStorage {
private:
    std::unordered_map<int, Employee> employees;

public:
    void save(const Employee& emp) override {
        employees[emp.getId()] = emp;
    }

    Employee* find(int id) override {
        auto it = employees.find(id);
        return (it != employees.end()) ? &it->second : nullptr;
    }

    std::vector<Employee> findAll() override {
        std::vector<Employee> result;
        for (auto& pair : employees) {
            result.push_back(pair.second);
        }
        return result;
    }
};
```

```
// File-based storage (for larger scale)
class FileEmployeeStorage : public IEmployeeStorage {
private:
    std::string directory;

public:
    FileEmployeeStorage(const std::string& dir) : directory(dir) {}

    void save(const Employee& emp) override {
        // Save to file - can handle more data than memory
        std::string filename = directory + "/employee_" +
                                std::to_string(emp.getId()) + ".txt";
        std::ofstream file(filename);
        // Save employee data to file...
    }

    Employee* find(int id) override {
        // Load from file on demand
        return nullptr; // Implementation
    }

    std::vector<Employee> findAll() override {
        return {}; // Implementation
    }
};

// Distributed storage (for massive scale)
class DistributedEmployeeStorage : public IEmployeeStorage {
    // Could connect to database clusters, distributed cache, etc.
public:
    void save(const Employee& emp) override {
        // Save to database cluster
    }
};
```

```
        std::cout << "Saving to distributed database cluster...\n";
    }

    Employee* find(int id) override {
        // Query from distributed database
        return nullptr;
    }

    std::vector<Employee> findAll() override {
        return {};
    }
};

// ===== SCALABLE EMPLOYEE SERVICE =====

class ScalableEmployeeService {
private:
    std::unique_ptr<IEmployeeStorage> storage;
    std::unique_ptr<ThreadPool> threadPool;

public:
    ScalableEmployeeService(std::unique_ptr<IEmployeeStorage> storage,
                            size_t numThreads = 4)
        : storage(std::move(storage)),
          threadPool(std::make_unique<ThreadPool>(numThreads)) {}

    void addEmployee(const Employee& emp) {
        storage->save(emp);
    }

    // Process multiple requests concurrently
    void processAsync(const std::function<void()>& task) {
```

```
        threadPool->enqueue(task);
    }

    // Scale by changing storage backend!
    void scaleToDistributed() {
        storage = std::make_unique<DistributedEmployeeStorage>();
    }
};

// ===== DEMONSTRATION =====
int main() {
    std::cout << "=== Scalability Demonstration ===\n\n";

    // Start with in-memory storage
    auto storage = std::make_unique<InMemoryEmployeeStorage>();
    ScalableEmployeeService service(std::move(storage), 4);

    // Add employees
    service.addEmployee(Employee("Alice", 101, "Engineering", 75000));
    service.addEmployee(Employee("Bob", 102, "Sales", 65000));
    service.addEmployee(Employee("Charlie", 103, "Engineering", 80000));

    // Process multiple requests concurrently
    for (int i = 0; i < 3; ++i) {
        service.processAsync([i] {
            std::cout << "Processing request " << i << " on thread\n";
        });
    }

    // When system grows, switch to distributed storage - no code changes!
    std::cout << "\n--- Scaling to distributed storage ---\n";
    // service.scaleToDistributed(); // Uncomment to switch
```

```
std::cout << "\nScalability features demonstrated:\n";  
std::cout << "1. O(1) indexing for fast lookups\n";  
std::cout << "2. Pagination for large datasets\n";  
std::cout << "3. Thread pool for concurrent requests\n";  
std::cout << "4. Interchangeable storage backends\n";  
  
return 0;  
}
```

Output:

```
=== Scalability Demonstration ===
```

```
Processing request 0 on thread
```

```
Processing request 1 on thread
```

```
Processing request 2 on thread
```

```
Scalability features demonstrated:
```

```
1. O(1) indexing for fast lookups
```

```
2. Pagination for large datasets
```

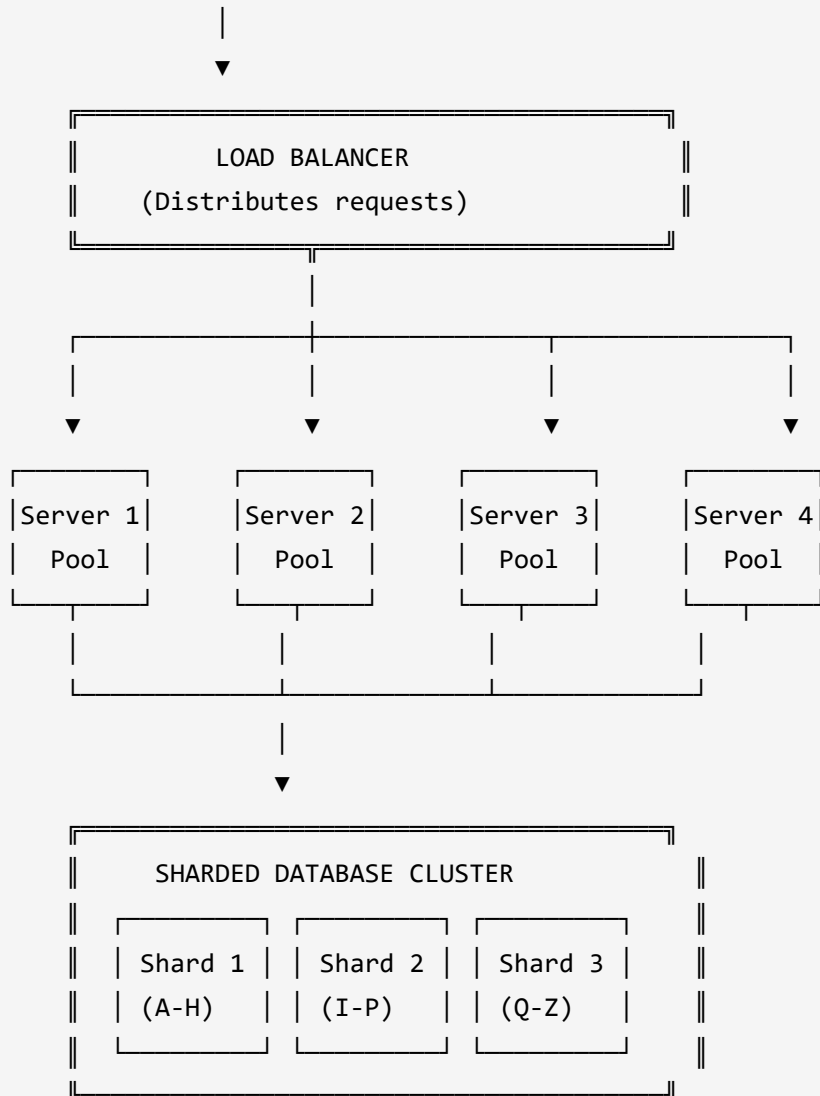
```
3. Thread pool for concurrent requests
```

```
4. Interchangeable storage backends
```


Visual Representation

SCALABILITY ARCHITECTURE

Layer 1: Client Requests



- ✓ Add more servers = handle more users
- ✓ Add more shards = store more data
- ✓ No single point of failure

Scalability Checklist

SCALABILITY DESIGN CHECKLIST

- ☐ Use indexing for fast lookups ($O(1)$ vs $O(n)$)
- ☐ Implement pagination for large datasets
- ☐ Use thread pools for concurrent processing
- ☐ Design for distributed/decentralized data
- ☐ Use caching for frequently accessed data
- ☐ Abstract storage layer for flexibility
- ☐ Design stateless services where possible
- ☐ Consider async processing for long operations

5b. Maintainability in Design

Definition

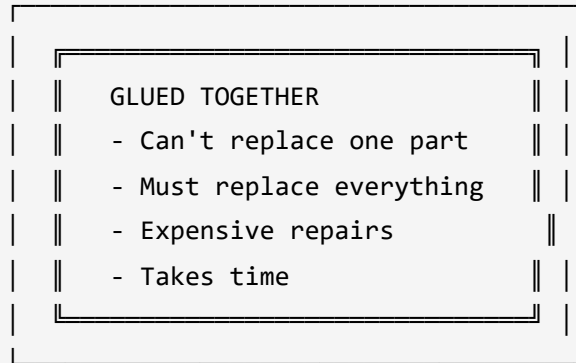
Maintainability is the ease with which software can be modified to:

- Fix bugs
- Add new features
- Improve performance
- Adapt to new environments

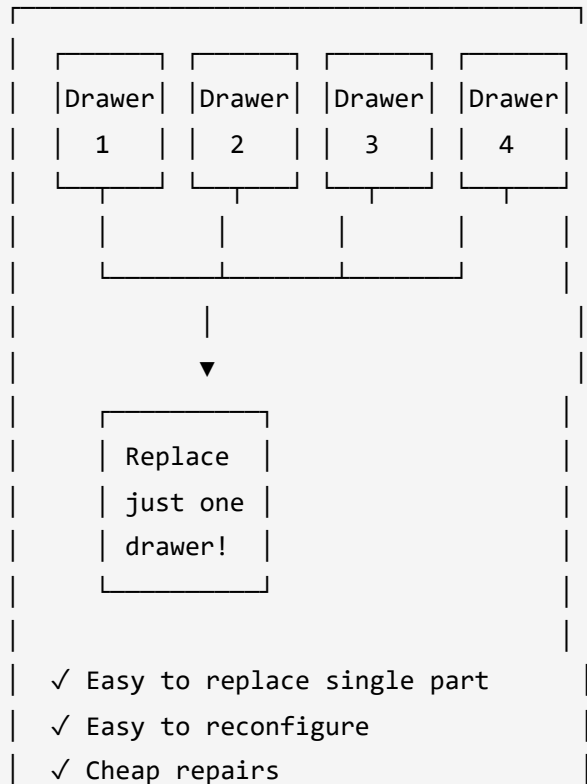
The Analogy

Think of a **Modular Furniture** vs **Built-in Cabinet**.

✗ BUILT-IN CABINET - Hard to maintain



✓ MODULAR FURNITURE - Easy to maintain



| ✓ Quick to fix |

Key Aspects of Maintainability

MAINTAINABILITY PILLARS

1. READABILITY

└─ Code is easy to understand

- Clear naming
- Good comments
- Simple structure

2. MODIFIABILITY

└─ Changes are easy to make

- Loose coupling
- High cohesion
- SOLID principles

3. TESTABILITY

└─ Easy to verify correctness

- Small, focused functions
- Dependency injection
- Mockable dependencies

4. DEBUGGABILITY

└─ Easy to find and fix problems

- Good error messages
- Logging
- Stack traces



Bad Example (Not Maintainable)

```
#include <iostream>
#include <string>
#include <vector>

// ❌ VIOLATION: Hard to maintain

class EmployeeManager {
private:
    // Single massive class with everything
    std::vector<std::string> names;
    std::vector<int> ids;
    std::vector<double> salaries;
    std::vector<std::string> departments;
    std::vector<std::string> emails;
    // 50 more fields...

    // Mixed concerns - everything in one place
    std::string databaseConnection; // Database logic
    std::string emailServer;        // Email logic
    std::string reportFormat;       // Report logic

public:
    // 200 methods all in one class!
    void add() { /* ... */ }
    void update() { /* ... */ }
    void delete() { /* ... */ }
    void saveToDatabase() { /* ... */ }
    void loadFromDatabase() { /* ... */ }
    void sendEmail() { /* ... */ }
```



```
void generateReport() { /* ... */ }
void exportPDF() { /* ... */ }
void exportCSV() { /* ... */ }
void calculateSalary() { /* ... */ }
void calculateBonus() { /* ... */ }
void calculateTax() { /* ... */ }
void validateEmail() { /* ... */ }
void validatePhone() { /* ... */ }
// 200 more methods...

// Hardcoded values everywhere
static constexpr double TAX_RATE = 0.20;
static constexpr double BONUS_RATE = 0.10;
// What if tax rates change?
// Have to change in 20 places!

// No clear structure - can't find anything
// Testing is impossible - everything coupled
// Adding features is risky - might break anything
};

// Problems:
// 1. 5000-line class - can't understand it all
// 2. Everything coupled - change one thing, break another
// 3. Can't test - too many dependencies
// 4. No clear ownership - who maintains what?
// 5. Hardcoded values scattered everywhere
// 6. No clear API - don't know where to start
```

Good Example (Maintainable)

```
#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <sstream>

//  GOOD: Maintainable design with clear separation

// ===== 1. SINGLE RESPONSIBILITY CLASSES =====

// Only holds employee data - pure data class
class Employee {
private:
    std::string name;
    int id;
    std::string department;
    double salary;

public:
    Employee(std::string n, int i, std::string d, double s)
        : name(n), id(i), department(d), salary(s) {}

    // Getters and setters only
    std::string getName() const { return name; }
    int getId() const { return id; }
    std::string getDepartment() const { return department; }
    double getSalary() const { return salary; }

    void setSalary(double s) { salary = s; }
```

```
};

// Configuration class - one place for all constants
class EmployeeConfig {
public:
    static constexpr double DEFAULT_BONUS_RATE = 0.10;
    static constexpr double DEFAULT_TAX_RATE = 0.20;
    static constexpr int MIN_SALARY = 0;
    static constexpr int MAX_SALARY = 1000000;
    static constexpr int MAX_NAME_LENGTH = 100;

    // Easy to change - single location!
};

// Validation in one place
class EmployeeValidator {
public:
    static bool isValidSalary(double salary) {
        return salary >= EmployeeConfig::MIN_SALARY &&
            salary <= EmployeeConfig::MAX_SALARY;
    }

    static bool isValidName(const std::string& name) {
        return !name.empty() &&
            name.length() <= EmployeeConfig::MAX_NAME_LENGTH;
    }

    static bool isValidEmployee(const Employee& emp) {
        return isValidName(emp.getName()) &&
            isValidSalary(emp.getSalary());
    }
};
```

```
// Calculations in one place
class SalaryCalculator {
private:
    double taxRate;
    double bonusRate;

public:
    SalaryCalculator(double tax = EmployeeConfig::DEFAULT_TAX_RATE,
                    double bonus = EmployeeConfig::DEFAULT_BONUS_RATE)
        : taxRate(tax), bonusRate(bonus) {}

    double calculateNetSalary(double gross) const {
        return gross * (1 - taxRate);
    }

    double calculateBonus(double salary) const {
        return salary * bonusRate;
    }

    // Easy to modify rates in one place!
    void updateTaxRate(double newRate) { taxRate = newRate; }
    void updateBonusRate(double newRate) { bonusRate = newRate; }
};

// ===== 2. DEPENDENCY INJECTION FOR TESTING =====

class IEmployeeRepository {
public:
    virtual void save(const Employee& emp) = 0;
    virtual Employee* find(int id) = 0;
    virtual std::vector<Employee> findAll() = 0;
```

```
    virtual ~IEmployeeRepository() = default;
};

// Easy to mock for testing!
class MockEmployeeRepository : public IEmployeeRepository {
private:
    std::vector<Employee> data;

public:
    void save(const Employee& emp) override {
        data.push_back(emp);
    }

    Employee* find(int id) override {
        for (auto& emp : data) {
            if (emp.getId() == id) return &emp;
        }
        return nullptr;
    }

    std::vector<Employee> findAll() override { return data; }
};

// ===== 3. LOGGING FOR DEBUGGING =====

class Logger {
public:
    enum class Level { DEBUG, INFO, WARNING, ERROR };

    static void log(Level level, const std::string& message) {
        std::string prefix;
        switch (level) {
```

```
        case Level::DEBUG:    prefix = "[DEBUG]"; break;
        case Level::INFO:     prefix = "[INFO]"; break;
        case Level::WARNING:  prefix = "[WARN]"; break;
        case Level::ERROR:    prefix = "[ERROR]"; break;
    }
    std::cout << prefix << " " << message << "\n";
}
};

// ===== 4. MAINTAINABLE EMPLOYEE SERVICE =====

class EmployeeService {
private:
    std::unique_ptr<IEmployeeRepository> repository;
    SalaryCalculator calculator;
    Logger logger;

public:
    EmployeeService(std::unique_ptr<IEmployeeRepository> repo,
                    double taxRate = EmployeeConfig::DEFAULT_TAX_RATE,
                    double bonusRate = EmployeeConfig::DEFAULT_BONUS_RATE)
        : repository(std::move(repo)),
          calculator(taxRate, bonusRate) {}

    bool addEmployee(const Employee& emp) {
        Logger::log(Logger::Level::INFO, "Adding employee: " + emp.getName());

        if (!EmployeeValidator::isValidEmployee(emp)) {
            Logger::log(Logger::Level::ERROR, "Invalid employee data");
            return false;
        }
    }
};
```

```
        repository->save(emp);
        Logger::log(Logger::Level::INFO, "Employee added successfully");
        return true;
    }

    std::string generatePayrollReport() {
        Logger::log(Logger::Level::DEBUG, "Generating payroll report");

        std::ostringstream report;
        auto employees = repository->findAll();

        for (const auto& emp : employees) {
            double netSalary = calculator.calculateNetSalary(emp.getSalary());
            double bonus = calculator.calculateBonus(emp.getSalary());

            report << emp.getName() << ": $" << netSalary
                << " (Bonus: $" << bonus << ")\n";
        }

        return report.str();
    }

    // Easy to add new functionality without breaking existing code
    // Clear API - easy to understand
    // Easy to test - dependencies are injected
    // Easy to debug - logging everywhere
};

// ===== DEMONSTRATION =====
int main() {
    std::cout << "=== Maintainability Demonstration ===\n\n";
```

```
// Use mock for testing, real repository in production
auto repo = std::make_unique<MockEmployeeRepository>();
EmployeeService service(std::move(repo));

// Add employees
service.addEmployee(Employee("Alice", 101, "Engineering", 75000));
service.addEmployee(Employee("Bob", 102, "Sales", 65000));
service.addEmployee(Employee("Charlie", 103, "Marketing", 55000));

// Generate report
std::cout << service.generatePayrollReport();

std::cout << "\nMaintainability features:\n";
std::cout << "✓ Single responsibility classes\n";
std::cout << "✓ Dependency injection for testing\n";
std::cout << "✓ Configuration in one place\n";
std::cout << "✓ Logging for debugging\n";
std::cout << "✓ Clear API and separation of concerns\n";

return 0;
}
```

Output:

=== Maintainability Demonstration ===

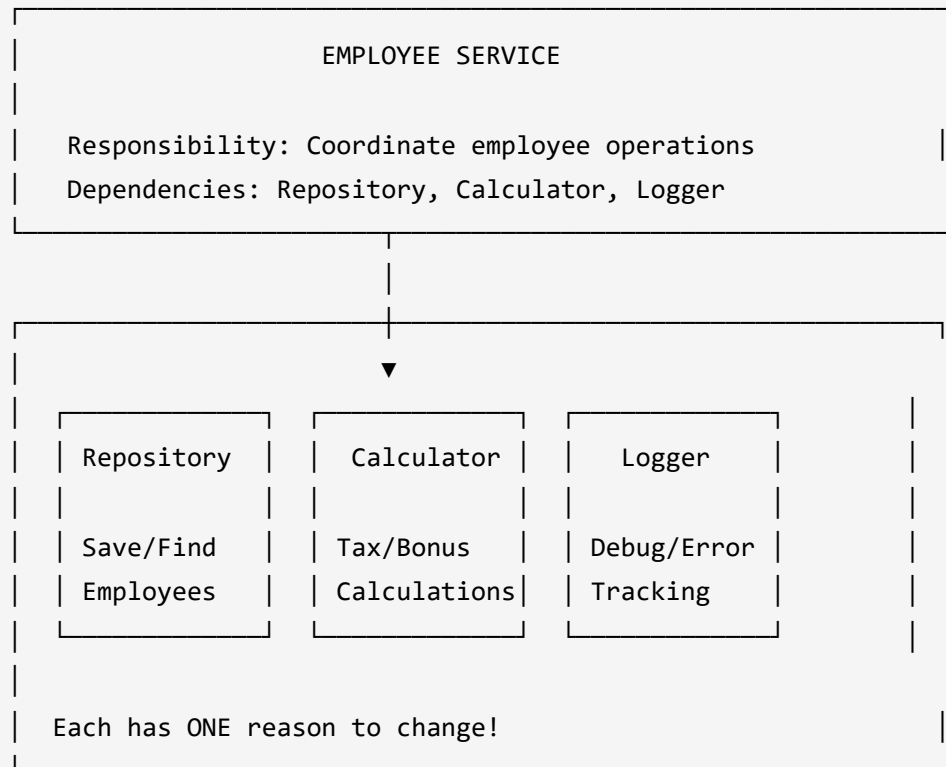
[INFO] Adding employee: Alice
[INFO] Employee added successfully
[INFO] Adding employee: Bob
[INFO] Employee added successfully
[INFO] Adding employee: Charlie
[INFO] Employee added successfully
[DEBUG] Generating payroll report
Alice: \$60000 (Bonus: \$7500)
Bob: \$52000 (Bonus: \$6500)
Charlie: \$44000 (Bonus: \$5500)

Maintainability features:

- ✓ Single responsibility classes
- ✓ Dependency injection for testing
- ✓ Configuration in one place
- + Logging for debugging
- + Clear API and separation of concerns

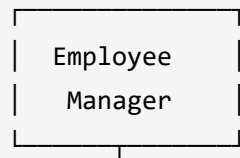
Visual Representation

MAINTAINABLE ARCHITECTURE

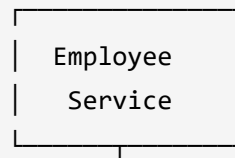


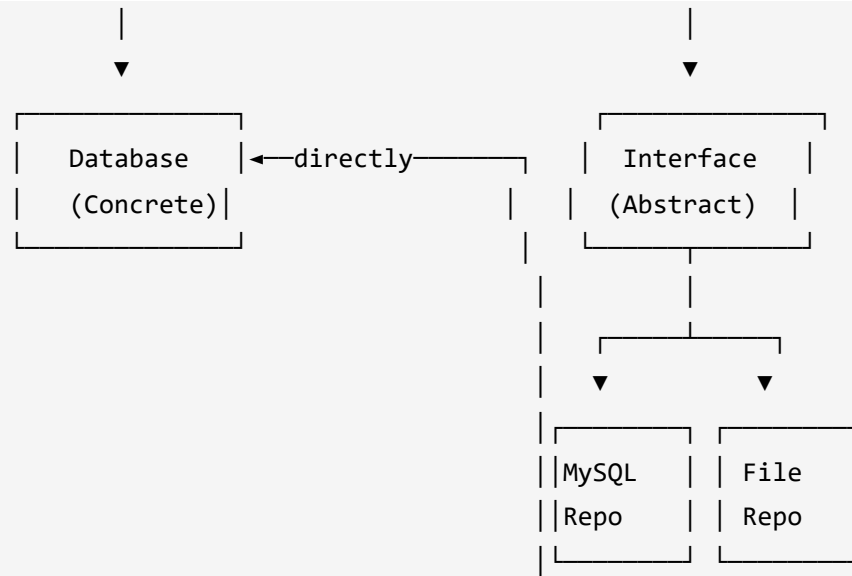
COUPLING DIAGRAM

TIGHT COUPLING (Not Maintainable)



LOOSE COUPLING (Maintainable)





Change DB = Change Manager

Can swap implementations
without changing Service

Maintainability Checklist

MAINTAINABILITY DESIGN CHECKLIST

- ☐ Use SOLID principles
- ☐ Keep classes small and focused
- ☐ Use dependency injection
- ☐ Centralize configuration
- ☐ Add logging for debugging
- ☐ Use interfaces for flexibility
- ☐ Write self-documenting code
- ☐ Keep functions short and simple
- ☐ Use meaningful names
- ☐ Write tests for all components

5c. Patterns and Techniques

Definition

Design Patterns are proven solutions to common software design problems. They represent best practices accumulated over decades of software development.

The Analogy

Think of **Recipe Cards** vs **Making It Up**.

✗ "MAKE IT UP" - Trial and error

"I think I'll add some salt..."

"Maybe some sugar?"

"Let me try this..."

Results: Unpredictable

Time: Wasted

Frustration: High

✓ "FOLLOW RECIPE" - Proven pattern

Recipe: "Chocolate Chip Cookies"

- 2 cups flour

- 1 cup sugar

- Follow steps 1-10

Results: Always works!

Time: Efficient

Quality: Consistent

Design patterns are "recipes" for code!

Common Patterns Overview

DESIGN PATTERNS CATALOG

CREATIONAL (How to create objects)

- Singleton - One instance of a class
- Factory - Create objects without specifying exact class
- Builder - Construct complex objects step-by-step

STRUCTURAL (How to organize classes)

- Adapter - Convert interface to another
- Facade - Simple interface to complex system
- Decorator - Add behavior dynamically

BEHAVIORAL (How objects communicate)

- Observer - Notify multiple objects of changes
- Strategy - Interchangeable algorithms
- Command - Encapsulate request as an object

Pattern 1: Factory Pattern

Purpose: Create objects without specifying exact class

```
#include <iostream>
#include <string>
#include <memory>

// ===== FACTORY PATTERN =====

// Product interface
class Employee {
public:
    virtual void work() = 0;
    virtual ~Employee() = default;
};

// Concrete products
class Engineer : public Employee {
public:
    void work() override {
        std::cout << "Engineer writing code...\n";
    }
};

class Manager : public Employee {
public:
    void work() override {
        std::cout << "Manager leading team...\n";
    }
};

class Intern : public Employee {
public:
    void work() override {
```



```
        std::cout << "Intern learning...\n";
    }
};

// Factory - creates objects without knowing exact type
class EmployeeFactory {
public:
    static std::unique_ptr<Employee> create(const std::string& type) {
        if (type == "engineer") {
            return std::make_unique<Engineer>();
        } else if (type == "manager") {
            return std::make_unique<Manager>();
        } else if (type == "intern") {
            return std::make_unique<Intern>();
        }
        return nullptr; // Unknown type
    }
};

// Usage - no need to know specific class!
int main() {
    std::cout << "=== Factory Pattern ===\n\n";

    auto emp1 = EmployeeFactory::create("engineer");
    auto emp2 = EmployeeFactory::create("manager");
    auto emp3 = EmployeeFactory::create("intern");

    emp1->work();
    emp2->work();
    emp3->work();

    // Easy to add new types - just add to factory!
```

```
std::cout << "\n✓ Add new employee types without changing client code\n";

return 0;
}
```

Output:

```
=== Factory Pattern ===
```

```
Engineer writing code...
```

```
Manager leading team...
```

```
Intern learning...
```

```
✓ Add new employee types without changing client code
```

Pattern 2: Strategy Pattern

Purpose: Interchangeable algorithms

```
#include <iostream>
#include <string>
#include <memory>
#include <functional>

// ===== STRATEGY PATTERN =====

// Strategy interface
class BonusStrategy {
public:
    virtual double calculate(double salary) = 0;
    virtual ~BonusStrategy() = default;
};

// Concrete strategies
class JuniorBonusStrategy : public BonusStrategy {
public:
    double calculate(double salary) override {
        return salary * 0.05; // 5%
    }
};

class SeniorBonusStrategy : public BonusStrategy {
public:
    double calculate(double salary) override {
        return salary * 0.15; // 15%
    }
};

class ExecutiveBonusStrategy : public BonusStrategy {
public:
```

```
double calculate(double salary) override {
    return salary * 0.25; // 25%
}

};

// Context - uses a strategy
class EmployeeBonus {
private:
    std::unique_ptr<BonusStrategy> strategy;

public:
    EmployeeBonus(std::unique_ptr<BonusStrategy> strat)
        : strategy(std::move(strat)) {}

    void setStrategy(std::unique_ptr<BonusStrategy> strat) {
        strategy = std::move(strat);
    }

    double calculate(double salary) {
        return strategy->calculate(salary);
    }
};

int main() {
    std::cout << "=== Strategy Pattern ===\n\n";

    EmployeeBonus bonus(std::make_unique<JuniorBonusStrategy>());
    std::cout << "Junior bonus on $50,000: $" << bonus.calculate(50000) << "\n";

    // Change strategy at runtime!
    bonus.setStrategy(std::make_unique<SeniorBonusStrategy>());
    std::cout << "Senior bonus on $50,000: $" << bonus.calculate(50000) << "\n";
}
```

```
bonus.setStrategy(std::make_unique<ExecutiveBonusStrategy>());  
std::cout << "Executive bonus on $50,000: $" << bonus.calculate(50000) << "\n";  
  
std::cout << "\n✓ Can change algorithms without changing client code\n";  
std::cout << "✓ Easy to add new strategies\n";  
  
return 0;  
}
```

Output:

```
=== Strategy Pattern ===
```

```
Junior bonus on $50,000: $2500
```

```
Senior bonus on $50,000: $7500
```

```
Executive bonus on $50,000: $12500
```

```
✓ Can change algorithms without changing client code
```

```
✓ Easy to add new strategies
```

Pattern 3: Observer Pattern

Purpose: Notify multiple objects of changes

```
#include <iostream>
#include <string>
#include <vector>
#include <memory>

// ===== OBSERVER PATTERN =====

// Observer interface
class IEmployeeObserver {
public:
    virtual void onEmployeeChanged(int id, const std::string& change) = 0;
    virtual ~IEmployeeObserver() = default;
};

// Subject being observed
class EmployeeSubject {
private:
    std::vector<std::unique_ptr<IEmployeeObserver>> observers;

public:
    void attachObserver(std::unique_ptr<IEmployeeObserver> observer) {
        observers.push_back(std::move(observer));
    }

    void notifyAll(int id, const std::string& change) {
        for (auto& observer : observers) {
            observer->onEmployeeChanged(id, change);
        }
    }
};
```

```
// Concrete observers
class PayrollSystem : public IEmployeeObserver {
public:
    void onEmployeeChanged(int id, const std::string& change) override {
        std::cout << "[Payroll] Employee " << id << ": " << change << "\n";
    }
};

class HRDepartment : public IEmployeeObserver {
public:
    void onEmployeeChanged(int id, const std::string& change) override {
        std::cout << "[HR] Employee " << id << ": " << change << "\n";
    }
};

class ManagementDashboard : public IEmployeeObserver {
public:
    void onEmployeeChanged(int id, const std::string& change) override {
        std::cout << "[Dashboard] Employee " << id << ": " << change << "\n";
    }
};

// Employee that notifies observers
class ObservableEmployee {
private:
    int id;
    std::string name;
    double salary;
    EmployeeSubject subject;

public:
    ObservableEmployee(int i, const std::string& n, double s)
```

```
        : id(i), name(n), salary(s) {}

void setSalary(double newSalary) {
    salary = newSalary;
    subject.notifyAll(id, "Salary changed to $" + std::to_string(salary));
}

void attachObserver(std::unique_ptr<IEmployeeObserver> observer) {
    subject.attachObserver(std::move(observer));
}

};

int main() {
    std::cout << "=== Observer Pattern ===\n\n";

    ObservableEmployee emp(101, "Alice", 75000);

    // Multiple observers watch the same employee
    emp.attachObserver(std::make_unique<PayrollSystem>());
    emp.attachObserver(std::make_unique<HRDepartment>());
    emp.attachObserver(std::make_unique<ManagementDashboard>());

    std::cout << "--- Changing salary ---\n";
    emp.setSalary(80000);

    std::cout << "\n✓ All observers notified automatically\n";
    std::cout << "✓ Easy to add new observers\n";
    std::cout << "✓ Decoupled - employee doesn't know about observers\n";

    return 0;
}
```


Output:

```
=== Observer Pattern ===

--- Changing salary ---
[Payroll] Employee 101: Salary changed to $80000
[HR] Employee 101: Salary changed to $80000
[Dashboard] Employee 101: Salary changed to $80000

✓ All observers notified automatically
+ Easy to add new observers
+ Decoupled - employee doesn't know about observers
```

Pattern 4: Decorator Pattern

Purpose: Add behavior dynamically

```
#include <iostream>
#include <string>
#include <memory>

// ===== DECORATOR PATTERN =====

// Base component
class EmployeeReport {
public:
    virtual std::string generate() = 0;
    virtual ~EmployeeReport() = default;
};

// Concrete component
class BasicReport : public EmployeeReport {
private:
    std::string employeeName;
    double salary;

public:
    BasicReport(const std::string& name, double s)
        : employeeName(name), salary(s) {}

    std::string generate() override {
        return "Employee: " + employeeName + "\nSalary: $" + std::to_string(salary) + "\n";
    }
};

// Base decorator
class ReportDecorator : public EmployeeReport {
protected:
```

```
std::unique_ptr<EmployeeReport> wrappedReport;

public:
    ReportDecorator(std::unique_ptr<EmployeeReport> report)
        : wrappedReport(std::move(report)) {}
};

// Concrete decorators - add new features!
class WithBonusDecorator : public ReportDecorator {
private:
    double bonus;

public:
    WithBonusDecorator(std::unique_ptr<EmployeeReport> report, double b)
        : ReportDecorator(std::move(report)), bonus(b) {}

    std::string generate() override {
        return wrappedReport->generate() + "Bonus: $" + std::to_string(bonus) + "\n";
    }
};

class WithTaxDecorator : public ReportDecorator {
private:
    double tax;

public:
    WithTaxDecorator(std::unique_ptr<EmployeeReport> report, double t)
        : ReportDecorator(std::move(report)), tax(t) {}

    std::string generate() override {
        return wrappedReport->generate() + "Tax: $" + std::to_string(tax) + "\n";
    }
}
```

```
};

class WithTimestampDecorator : public ReportDecorator {
public:
    WithTimestampDecorator(std::unique_ptr<EmployeeReport> report)
        : ReportDecorator(std::move(report)) {}

    std::string generate() override {
        return wrappedReport->generate() + "Generated: 2024-01-15\n";
    }
};

int main() {
    std::cout << "=== Decorator Pattern ===\n\n";

    // Basic report
    auto report = std::make_unique<BasicReport>("Alice", 75000);
    std::cout << "--- Basic Report ---\n";
    std::cout << report->generate();

    // Add bonus
    report = std::make_unique<WithBonusDecorator>(std::move(report), 5000);
    std::cout << "\n--- With Bonus ---\n";
    std::cout << report->generate();

    // Add tax
    report = std::make_unique<WithTaxDecorator>(std::move(report), 15000);
    std::cout << "\n--- With Tax ---\n";
    std::cout << report->generate();

    // Add timestamp
    report = std::make_unique<WithTimestampDecorator>(std::move(report));
```

```
std::cout << "\n--- Complete Report ---\n";  
std::cout << report->generate();  
  
std::cout << "\n✓ Can add features without modifying original class\n";  
std::cout << "✓ Can combine decorators in different ways\n";  
std::cout << "✓ Open/Closed Principle in action!\n";  
  
return 0;  
}
```

Output:

=== Decorator Pattern ===

--- Basic Report ---

Employee: Alice

Salary: \$75000

--- With Bonus ---

Employee: Alice

Salary: \$75000

Bonus: \$5000

--- With Tax ---

Employee: Alice

Salary: \$75000

Bonus: \$5000

Tax: \$15000

--- Complete Report ---

Employee: Alice

Salary: \$75000

Bonus: \$5000

Tax: \$15000

Generated: 2024-01-15

✓ Can add features without modifying original class

✓ Can combine decorators in different ways

+ Open/Closed Principle in action!

Pattern 5: Facade Pattern

Purpose: Simple interface to complex system

```
#include <iostream>
#include <string>
#include <vector>
#include <memory>

// ===== FACADE PATTERN =====

// Complex subsystem classes (don't need to know about these)
class DatabaseConnection {
public:
    void connect() { std::cout << "Connecting to database...\n"; }
    void query(const std::string& sql) { std::cout << "Executing: " << sql << "\n"; }
};

class EmailService {
public:
    void send(const std::string& to, const std::string& msg) {
        std::cout << "Sending email to " << to << ": " << msg << "\n";
    }
};

class ReportGenerator {
public:
    void generatePDF() { std::cout << "Generating PDF report...\n"; }
    void generateExcel() { std::cout << "Generating Excel report...\n"; }
};

class Logger {
public:
    void log(const std::string& msg) { std::cout << "[LOG] " << msg << "\n"; }
};
```



```
// Complex subsystem - employee management with many parts
class ComplexEmployeeSystem {
private:
    DatabaseConnection db;
    EmailService email;
    ReportGenerator report;
    Logger logger;

public:
    void hireEmployee(const std::string& name) {
        logger.log("Starting hire process for " + name);
        db.connect();
        db.query("INSERT INTO employees VALUES ('" + name + "')");
        email.send(name, "Welcome to the company!");
        logger.log("Hire complete for " + name);
    }

    void generateReport(const std::string& format) {
        logger.log("Generating " + format + " report");
        db.connect();
        db.query("SELECT * FROM employees");
        if (format == "pdf") {
            report.generatePDF();
        } else {
            report.generateExcel();
        }
    }

    // Many more complex operations...
};
```

```
// FACADE - simple interface to complex system
class EmployeeFacade {
private:
    ComplexEmployeeSystem system;

public:
    void hireEmployee(const std::string& name) {
        system.hireEmployee(name);
    }

    void generateReport(const std::string& format) {
        system.generateReport(format);
    }

    // Just two simple methods - complexity hidden!
};

int main() {
    std::cout << "=== Facade Pattern ===\n\n";

    // Client only sees the simple facade
    EmployeeFacade facade;

    std::cout << "--- Hiring employee ---\n";
    facade.hireEmployee("Alice");

    std::cout << "\n--- Generating PDF report ---\n";
    facade.generateReport("pdf");

    std::cout << "\n✓ Simple interface to complex subsystem\n";
    std::cout << "✓ Client doesn't need to know internal complexity\n";
    std::cout << "✓ Easy to use and understand\n";
}
```

```
    return 0;  
}
```

Output:

```
=== Facade Pattern ===
```

```
--- Hiring employee ---
```

```
[LOG] Starting hire process for Alice
```

```
Connecting to database...
```

```
Executing: INSERT INTO employees VALUES ('Alice')
```

```
Sending email to Alice: Welcome to the company!
```

```
[LOG] Hire complete for Alice
```

```
--- Generating PDF report ---
```

```
[LOG] Generating pdf report
```

```
Connecting to database...
```

```
Executing: SELECT * FROM employees
```

```
Generating PDF report
```

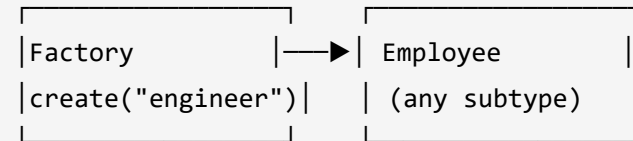
- ✓ Simple interface to complex subsystem
- ✓ Client doesn't need to know internal complexity
- + Easy to use and understand

Visual Pattern Summary

PATTERNS AT A GLANCE

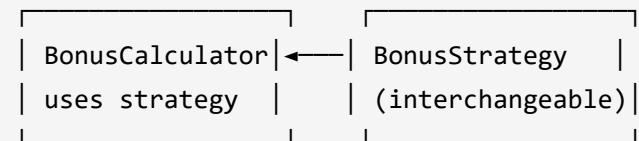
FACTORY

"I need an employee, give me whatever type you have"



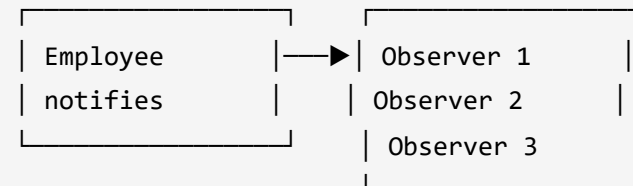
STRATEGY

"I need to calculate bonus, here's HOW to do it"



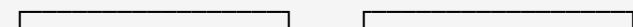
OBSERVER

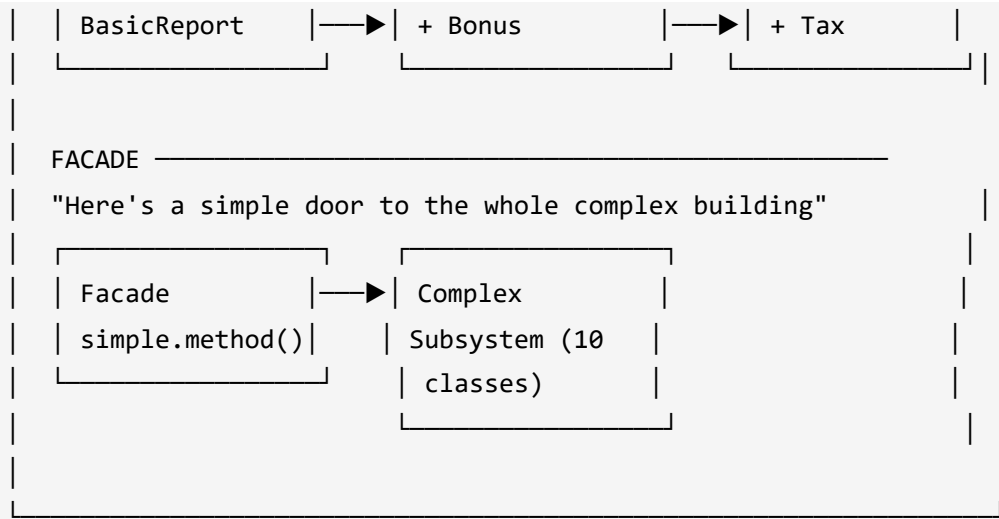
"When employee changes, tell ALL interested parties"



DECORATOR

"Add features LAYER BY LAYER without changing original"





SUMMARY

Quick Reference Card

SOLID PRINCIPLES

S - Single Responsibility	One class = one job
O - Open/Closed	Open for extension, Closed for modification
L - Liskov Substitution	Subclasses can replace parent classes
I - Interface Segregation	Many specific interfaces better than one general
D - Dependency Inversion	Depend on abstractions, not concretions

DRY - Don't Repeat Yourself	One piece of knowledge, one authoritative location
KISS - Keep It Simple	Simple > Complex If it's complicated, it's probably wrong

Remember

Code is read more than it is written.
Write for the developer who will maintain your code.
(It might be future-you!)

Good design principles lead to:

- Faster feature development
- Fewer bugs
- Easier debugging
- Happier developers
- Reduced technical debt

Apply these principles from the start,
not as an afterthought!

Document generated for educational purposes.

All examples use C++ and the Employee class for consistency and clarity.